

LIVLUXE

# Partner Direct Booking Platform

## End-to-End Architecture & Engineering Specification

A bespoke, invite-only booking portal for insurance and corporate partners. Inventory, availability, rates and check-in data synchronised from Hostaway. Payments, authorisations and invoicing through Stripe. Identity, application data and audit trail on Supabase. Deployed on Vercel.

|                      |                                                        |
|----------------------|--------------------------------------------------------|
| Document             | Architecture & Build Specification                     |
| Version              | 1.0 — Issued for build                                 |
| Date                 | 16 September 2026                                      |
| Audience             | Implementing software engineer / delivery team         |
| Primary integrations | Hostaway Public API v1 • Stripe API                    |
| Platform             | Next.js on Vercel • Supabase (Postgres, Auth, Storage) |
| Status               | Draft for technical review and estimation              |

This document is a complete build specification. It is written so that a competent full-stack engineer can implement the system without further design input, and so that a reviewer can verify that every functional and non-functional requirement has been addressed. Sections 17 and 23 list the decisions Livluxe must confirm before build starts.

# Contents

---

|    |                                                                                                                              |
|----|------------------------------------------------------------------------------------------------------------------------------|
| 1  | Product context, goals and scope                                                                                             |
| 2  | Users, roles and permissions                                                                                                 |
| 3  | System architecture                                                                                                          |
| 4  | Technology stack and rationale                                                                                               |
| 5  | Environments, repository and configuration                                                                                   |
| 6  | Hostaway integration                                                                                                         |
|    | 6.1 Authentication · 6.2 Rate limits · 6.3 Endpoints · 6.4 Sync strategy · 6.5 Webhooks · 6.6 Field mapping · 6.7 Write-back |
| 7  | Data model and database schema                                                                                               |
| 8  | Authentication, tenancy and access control                                                                                   |
| 9  | Rate, quote and pricing engine                                                                                               |
| 10 | Booking request lifecycle and approval workflow                                                                              |
| 11 | Stripe integration, invoicing and reconciliation                                                                             |
| 12 | Notifications, messaging and comment threads                                                                                 |
| 13 | Frontend application specification                                                                                           |
| 14 | Map and heat map specification                                                                                               |
| 15 | Search, filtering and availability queries                                                                                   |
| 16 | Internal API surface                                                                                                         |
| 17 | Edge cases and failure modes                                                                                                 |
| 18 | Security, privacy and compliance                                                                                             |
| 19 | Observability, alerting and runbooks                                                                                         |
| 20 | Testing and quality assurance                                                                                                |
| 21 | Deployment, CI/CD and cutover                                                                                                |
| 22 | Delivery plan and estimates                                                                                                  |
| 23 | Risks, assumptions and open decisions                                                                                        |
| A  | Environment variables                                                                                                        |
| B  | Booking state transition table                                                                                               |
| C  | Webhook event catalogue                                                                                                      |
| D  | Email and notification templates                                                                                             |
| E  | Glossary                                                                                                                     |

## How to read this document

Sections 1–5 establish context and the shape of the system. Sections 6–12 are the integration and domain core — these contain the logic that is expensive to get wrong. Sections 13–16 describe the surfaces (UI and API). Sections 17–21 cover resilience and operations. Sections 22–23 cover delivery.

Where a decision has been made on Livlux's behalf, it is marked as a **recommendation** with the reasoning attached, so it can be overridden deliberately rather than by accident. Anything that requires a business answer is collected in Section 23 rather than left implicit in the text.

#### VERIFY BEFORE BUILDING

Hostaway ships no public sandbox and its API has changed materially in the past (the legacy webhook modules were sunset, and reservation fields are added regularly). Every endpoint, field name and payload shape in Section 6 must be re-verified against <https://api.hostaway.com/documentation> and against a live Hostaway test account before code is written against it. Treat this document as the architecture of record and the Hostaway docs as the schema of record.

# 1 Product context, goals and scope

## 1.1 The business problem

Livlux operates a portfolio of short-term rental properties managed in Hostaway. A meaningful share of demand does not come from OTA guests at all — it comes from **insurance partners** placing displaced policyholders (fire, flood, storm, escape-of-water claims) and **corporate partners** placing relocating staff, project crews and contractors. These bookings have a different shape to OTA bookings:

- **Long stays.** Typically 14 nights to 6 months, not 2–3 nights. Rates are negotiated, not retail.
- **Deferred decision-making.** The partner enquires, the adjuster or relocation manager approves internally, and then the booking is confirmed. A "book now" flow does not fit.
- **Approval on the Livlux side too.** Not every request should be auto-accepted — suitability, cleaning turnarounds, owner blocks and commercial terms all matter.
- **Invoices, not receipts.** Partners require a tax invoice with claim numbers, purchase order references and cost centres for their own reimbursement process.
- **Repeat, high-trust relationships.** Partners are known entities with named users, not anonymous traffic.

Today this is transacted over email and phone: availability is quoted manually from Hostaway, rates are retyped, double-bookings are a live risk, and there is no self-service view of what is actually free. The platform replaces that with a permissioned portal.

## 1.2 What the platform is

An invite-only web portal where an authenticated partner can see live Livlux availability on a map and in a list, inspect a property in full detail, build a priced request for a long stay, submit it for approval, pay or be invoiced through Stripe, and track the booking through to check-in — while Livlux operations approve, price-adjust, invoice and push the confirmed reservation back into Hostaway as the single source of truth for the calendar.

## 1.3 Goals and success criteria

| Goal                                  | Description                                                                                          | Measurable success criterion                                                                       |
|---------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Eliminate manual availability quoting | Partners self-serve accurate, live availability and indicative pricing without contacting Livlux.    | >70% of partner bookings originate as a portal request within 3 months of launch.                  |
| Zero double-bookings                  | No confirmed partner booking ever collides with an existing Hostaway reservation or owner block.     | Zero calendar collisions attributable to the platform; all near-misses logged and alerted.         |
| Faster time to confirm                | Compress enquiry→confirmation from days to hours.                                                    | Median request-to-decision under 4 business hours; 95th percentile under 24 hours.                 |
| Clean money trail                     | Every booking has an auditable payment or invoice, a timestamp and an approver.                      | 100% of confirmed bookings reconcile to a Stripe object; monthly reconciliation variance = \$0.    |
| Hostaway remains the source of truth  | The portal never becomes a second, divergent calendar.                                               | Nightly reconciliation drift report shows zero unexplained differences.                            |
| Partner-grade presentation            | The product reflects Livlux's positioning and can be shown to a national insurer's procurement team. | WCAG 2.2 AA; Lighthouse performance ≥ 90 on listing search; passes partner security questionnaire. |

## 1.4 In scope

- Partner organisations with multiple named users, invite-only.
- Full listing catalogue synced from Hostaway, with photos, amenities, bedroom/bathroom/bed configuration, capacity, location and description.
- Live availability calendar per listing, with minimum-stay rules.
- Nightly rates from Hostaway plus partner-specific negotiated rate cards and long-stay discounts.
- Check-in and access details: check-in/out windows, access method, door/lock codes, parking, Wi-Fi, house rules — released on a controlled schedule.
- Map view with clustering and a configurable heat map layer.
- Multi-property request baskets (a partner placing five families at once).
- Quote engine with fees, taxes and a frozen price for a defined window.
- Booking request workflow with Livluxe approval, decline, counter-offer and expiry.
- Stripe payment: card authorisation and capture, BECS/ACH debit, and Stripe Invoicing for net-terms partners.
- Automatic invoice generation to the partner with claim/PO references.
- Approval notifications and timestamps at every state transition.
- Comment threads per booking request between partner and Livluxe.
- Write-back of confirmed bookings to Hostaway as reservations, and calendar holds during approval.
- Admin console for operations: request queue, rate cards, partner management, reporting, audit log.
- Document storage (certificates of currency, authority-to-book letters, PO documents).

## 1.5 Out of scope for v1

| Excluded                                            | Reasoning and future path                                                                                                                                                                                               |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Public consumer-facing booking site                 | Different funnel, different pricing, different SEO requirements. The architecture does not preclude it — the listing/quote/availability layer is reusable — but the partner portal must not be compromised to serve it. |
| Guest-facing mobile app                             | Responsive web covers the partner use case. Revisit only if field adjusters demand offline capability.                                                                                                                  |
| Two-way guest messaging with Hostaway unified inbox | Partner ↔ Livluxe comment threads are in scope; guest messaging stays in Hostaway. Hostaway's message webhook is noted in Appendix C for a later phase.                                                                 |
| Dynamic/revenue-managed pricing                     | The portal reads whatever rate Hostaway holds and applies the partner's contracted adjustment. Any RMS integration (Pricelabs, Wheelhouse) continues to act on Hostaway upstream.                                       |
| Owner statements and trust accounting               | Remains in Hostaway / existing accounting stack. The platform exports data but does not become a ledger.                                                                                                                |
| Automated ID verification of end-guests             | Partner is the counterparty and carries that responsibility contractually.                                                                                                                                              |
| Multi-currency                                      | v1 is AUD only. The schema stores currency per amount so this is a configuration change later, not a migration.                                                                                                         |

## 1.6 Guiding architectural principles

| Principle                                                    | What it means in practice                                                                                                                                                                       |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hostaway is the source of truth for inventory                | The platform holds a <i>cache</i> , never a master. Any conflict resolves in Hostaway's favour. Availability is re-verified against Hostaway at the moment of approval, not trusted from cache. |
| Money is never inferred                                      | Stripe is the source of truth for payment state. The database records intent and outcome, reconciled from Stripe webhooks — never from client-side success callbacks.                           |
| Every state change is an event with an actor and a timestamp | Appending to an immutable history table is mandatory, not optional. "Who approved this and when" must be answerable years later.                                                                |
| Fail closed on availability                                  | If the system cannot confirm a property is free, it does not offer it. Showing fewer properties is always cheaper than one double-booking.                                                      |

| Principle                                             | What it means in practice                                                                                                   |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Tenancy enforced at the database, not the application | Postgres row-level security is the boundary. An application bug must not be able to leak Partner A's bookings to Partner B. |
| Idempotency everywhere external                       | Every inbound webhook and every outbound write to Hostaway or Stripe carries an idempotency key and is safe to replay.      |
| Boring, hosted infrastructure                         | Vercel + Supabase + Stripe. No self-managed queues, no Kubernetes. The team that maintains this is small.                   |

## 2 Users, roles and permissions

### 2.1 Personas

| Persona                                       | Organisation      | Needs and behaviour                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claims consultant / accommodation coordinator | Insurance partner | Places a displaced family at short notice, often out of hours, often mid-phone-call with the policyholder. Needs: filter by suburb and bedrooms, see what is free tonight, see a total cost for 30 nights, submit with a claim number, get confirmation fast. Low tolerance for friction. May be on a tablet in the field. |
| Relocation / mobility manager                 | Corporate partner | Places staff for 1–6 months, plans weeks ahead, often books several units at once. Needs: compare properties, export a shortlist for internal approval, attach a PO number, receive a consolidated invoice, extend stays.                                                                                                  |
| Partner administrator                         | Partner           | Manages their own team's access: invites and removes colleagues, sets who may submit versus who may only view, maintains billing contact and PO defaults. Should never need to call Livlux to add a user.                                                                                                                  |
| Partner finance                               | Partner           | Read-only on bookings, full access to invoices and payment history. Downloads tax invoices and remittance data.                                                                                                                                                                                                            |
| Livlux operations                             | Livlux            | Reviews the request queue, checks suitability and cleaning turnarounds, approves/declines/counter-offers, releases check-in details, handles extensions and cancellations. The primary daily user of the admin console.                                                                                                    |
| Livlux finance / director                     | Livlux            | Rate cards, partner commercial terms, credit limits, invoicing, refunds, reconciliation and reporting. Approves anything above a value threshold.                                                                                                                                                                          |
| System / integration                          | —                 | Scheduled sync jobs, webhook consumers and reconciliation processes. Acts with a service identity; all of its writes are attributed to <code>actor_type='system'</code> in the audit log.                                                                                                                                  |

### 2.2 Role model

Two role dimensions exist and must not be conflated: a **platform role** (`partner` or `livlux`) and an **organisation role** within that side. A user belongs to exactly one organisation. Livlux staff belong to the internal organisation.

| Role                         | Side    | Capability summary                                                                                                                               |
|------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>partner_admin</code>   | Partner | Everything a booker can do, plus user management, org profile, default billing/PO settings, and visibility of the whole organisation's bookings. |
| <code>partner_booker</code>  | Partner | Search, quote, submit requests, comment, pay, view own and (configurable) org bookings, download invoices.                                       |
| <code>partner_viewer</code>  | Partner | Read-only: availability, listings, org bookings. Cannot submit or pay. Used for auditors and junior staff.                                       |
| <code>partner_finance</code> | Partner | Read-only bookings; full invoice and payment history access; can update billing contact.                                                         |
| <code>livlux_ops</code>      | Livlux  | Request queue, approve/decline/counter within value limit, release check-in details, manage holds, comment, trigger resync.                      |
| <code>livlux_finance</code>  | Livlux  | Ops capabilities plus rate cards, credit terms, invoicing, refunds, reconciliation, reporting.                                                   |
| <code>livlux_admin</code>    | Livlux  | Full access including partner creation, role assignment, feature flags, integration settings, audit log export, support impersonation.           |

## 2.3 Permission matrix

| Capability                       | P Admin | P Booker | P Viewer | P Finance | LL Ops      | LL Finance | LL Admin |
|----------------------------------|---------|----------|----------|-----------|-------------|------------|----------|
| View listings & availability     | ✓       | ✓        | ✓        | ✓         | ✓           | ✓          | ✓        |
| View partner-specific rates      | ✓       | ✓        | ✓        | ✓         | ✓           | ✓          | ✓        |
| Generate quote                   | ✓       | ✓        | —        | —         | ✓           | ✓          | ✓        |
| Submit booking request           | ✓       | ✓        | —        | —         | ✓*          | ✓*         | ✓*       |
| Pay / authorise card             | ✓       | ✓        | —        | —         | —           | ✓          | ✓        |
| View own org bookings            | ✓       | ✓        | ✓        | ✓         | ✓           | ✓          | ✓        |
| View other orgs' bookings        | —       | —        | —        | —         | ✓           | ✓          | ✓        |
| Comment on request               | ✓       | ✓        | —        | —         | ✓           | ✓          | ✓        |
| Approve / decline request        | —       | —        | —        | —         | ✓†          | ✓          | ✓        |
| Issue counter-offer              | —       | —        | —        | —         | ✓           | ✓          | ✓        |
| Release check-in details         | —       | —        | —        | —         | ✓           | ✓          | ✓        |
| View check-in details (released) | ✓       | ✓        | ✓        | —         | ✓           | ✓          | ✓        |
| Cancel booking                   | ✓‡      | ✓‡       | —        | —         | ✓           | ✓          | ✓        |
| Download invoice                 | ✓       | ✓        | —        | ✓         | ✓           | ✓          | ✓        |
| Issue refund                     | —       | —        | —        | —         | —           | ✓          | ✓        |
| Manage org users                 | ✓       | —        | —        | —         | —           | —          | ✓        |
| Manage rate cards                | —       | —        | —        | —         | —           | ✓          | ✓        |
| Create / suspend partner org     | —       | —        | —        | —         | —           | —          | ✓        |
| View audit log                   | —       | —        | —        | —         | Own actions | ✓          | ✓        |
| Impersonate user (support)       | —       | —        | —        | —         | —           | —          | ✓        |
| Trigger manual Hostaway resync   | —       | —        | —        | —         | ✓           | ✓          | ✓        |

\* Livlux staff may create a request *on behalf of* a partner (phone bookings); the request records both `created_by` and `on_behalf_of_org_id`.

† Subject to a configurable value ceiling per role; above it, escalation to `livlux_finance` is forced. ‡ Partner-initiated cancellation is permitted only before a configurable cut-off and always generates a Livlux notification; after cut-off it becomes a cancellation *request*.

### IMPLEMENTATION NOTE

Encode this matrix once, in the database, as a `role_permissions` lookup table seeded by migration — not as scattered `if (role === 'partner_admin')` checks in the UI. The UI reads permissions from the session; RLS policies and server-side route guards enforce them independently. Three layers (UI affordance, server route guard, RLS policy) is correct, not redundant.

## 3 System architecture

### 3.1 High-level view

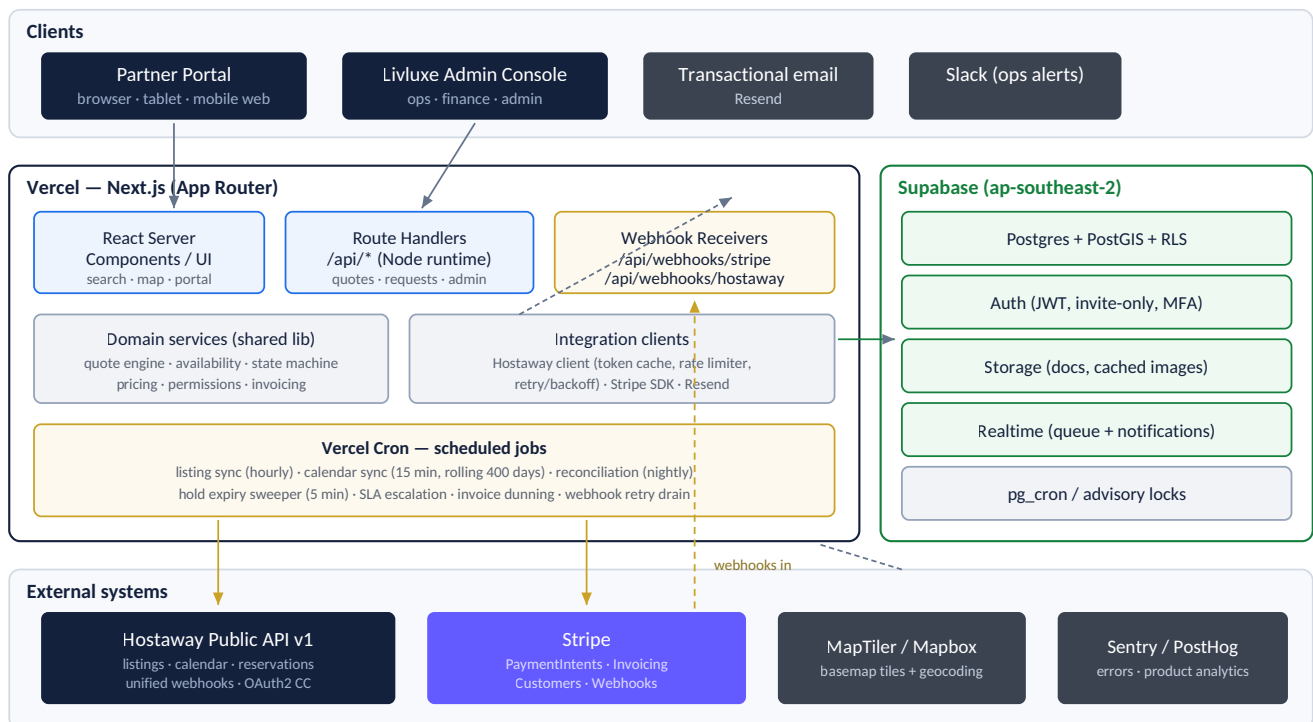


Figure 1 — System context and component architecture

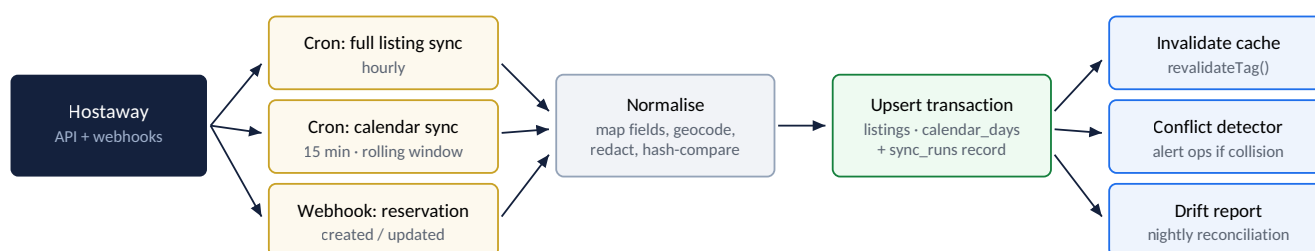
### 3.2 Component responsibilities

| Component                 | Responsibility                                                                                                                                              | Must not                                                                                                        |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Next.js RSC pages         | Render portal and admin UI. Read data through a Supabase client bound to the user's JWT, so RLS applies.                                                    | Hold service-role keys; call Hostaway or Stripe directly.                                                       |
| Route Handlers ( /api/* ) | All mutations: quoting, request submission, approval, payment intent creation, admin actions. Validate input with Zod, check permissions, write audit rows. | Run on the Edge runtime (the Stripe and Hostaway clients and raw-body signature checks need Node).              |
| Webhook receivers         | Verify authenticity, persist the raw event, return 2xx quickly, then process asynchronously and idempotently.                                               | Do slow work before responding; trust payload contents without re-fetching from the source API.                 |
| Domain services           | Pure, unit-testable logic: quote calculation, availability resolution, state machine transitions, permission evaluation, invoice composition.               | Import <code>next/headers</code> or any request-scoped context — keep them framework-free so they are testable. |
| Hostaway client           | Token acquisition and caching, global rate limiting, retry with exponential backoff and jitter, response normalisation, structured error types.             | Be called from more than one place without going through the limiter.                                           |

| Component             | Responsibility                                                                                                         | Must not                                                                                             |
|-----------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Scheduled jobs</b> | Sync, reconcile, expire holds, escalate SLAs, drain retry queues, send digests.                                        | Overlap with themselves — every job takes a Postgres advisory lock before doing work.                |
| <b>Postgres</b>       | System of record for platform data; enforces tenancy via RLS; enforces non-overlap of holds via exclusion constraints. | Be treated as a cache that can be rebuilt casually — the booking and audit tables are irreplaceable. |

### 3.3 The two critical data flows

#### 3.3.1 Inventory synchronisation (Hostaway → platform)



Three independent triggers converge on one idempotent upsert path. A webhook never writes a different code path to a cron sync.

Figure 2 — Inventory synchronisation pipeline

#### 3.3.2 Booking request (platform → Hostaway + Stripe)

Covered in detail in Sections 10 and 11 with a full sequence diagram. In summary: a quote is frozen, a *soft hold* is placed in the platform database (not in Hostaway), a card authorisation is taken without capture, Livlux approves, the system re-verifies availability against Hostaway live, creates the reservation in Hostaway, captures the payment, issues the invoice, and releases check-in details on schedule. Every one of those steps is individually retryable and individually reversible.

### 3.4 Why not a queue, a microservice split, or a separate backend?

| Considered                                       | Decision and reasoning                                                                                                                                                                                                                                                             |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Separate Node/NestJS backend                     | <b>Rejected.</b> Adds a deployment target, a second auth surface and CORS complexity for no benefit at this scale. Next.js Route Handlers on the Node runtime give the same capability with one deployment.                                                                        |
| Dedicated queue (SQS, BullMQ, Redis)             | <b>Deferred.</b> Webhook events are persisted to a Postgres table and drained by a cron worker with <code>FOR UPDATE SKIP LOCKED</code> . This is a durable queue with no extra infrastructure. Revisit if sustained throughput exceeds a few hundred events/minute — it will not. |
| Microservice per integration                     | <b>Rejected.</b> Two integrations, one team. Module boundaries inside one codebase give the same isolation without distributed-systems tax.                                                                                                                                        |
| Storing availability only in Hostaway (no cache) | <b>Rejected.</b> A map view over 100+ listings across a 6-month window would require hundreds of calendar calls per page load against a 15-request-per-10-second rate limit. A local cache is mandatory; correctness is preserved by re-verifying at approval time.                |
| Supabase Edge Functions for webhooks             | <b>Optional.</b> Viable, and worth using if Vercel function duration becomes a constraint on the nightly reconciliation. Start in Vercel for a single deployment story and single log stream.                                                                                      |

## 4 Technology stack and rationale

| Layer              | Choice                                                                                                                                                         | Rationale / notes                                                                                                                                                                        |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework          | Next.js (App Router, latest stable) + TypeScript in strict mode                                                                                                | Server Components keep partner data server-side by default; Route Handlers cover the API; one deployment unit on Vercel. Strict TS is non-negotiable given money and dates are involved. |
| UI                 | Tailwind CSS + shadcn/ui + Radix primitives + lucide-react                                                                                                     | Accessible primitives out of the box (dialog, combobox, date range picker), themable to Livlux brand, no heavyweight component library lock-in.                                          |
| Forms & validation | react-hook-form + Zod, with the <i>same</i> Zod schemas reused server-side                                                                                     | One schema definition validates client and server. Never trust the client copy.                                                                                                          |
| Data layer         | Supabase Postgres 15+, PostGIS enabled, generated TypeScript types                                                                                             | <code>supabase gen types typescript</code> in CI keeps types honest against migrations.                                                                                                  |
| Auth               | Supabase Auth — email invite + password, magic link fallback, TOTP MFA required for Livlux roles                                                               | JWT carries <code>org_id</code> and <code>role</code> as custom claims, which RLS policies read directly.                                                                                |
| Authorisation      | Postgres RLS + server-side guards + UI gating                                                                                                                  | See §8. RLS is the boundary that actually matters.                                                                                                                                       |
| Payments           | Stripe — PaymentIntents (manual capture), Setup Intents, Invoicing, Customers, Stripe Tax (optional)                                                           | Stripe Elements keeps card data off Livlux infrastructure → PCI DSS SAQ-A. See §11.                                                                                                      |
| PMS integration    | Hostaway Public API v1, OAuth2 client credentials, unified webhooks                                                                                            | Hand-rolled typed client (no official SDK exists). See §6.                                                                                                                               |
| Maps               | MapLibre GL JS + MapTiler or Mapbox vector tiles; <code>supercluster</code> for clustering; <code>deck.gl HeatmapLayer</code> or MapLibre native heatmap layer | MapLibre avoids per-map-load licensing surprises; tile provider is swappable. See §14.                                                                                                   |
| Email              | Resend + React Email templates                                                                                                                                 | Templates as versioned React components, previewed in Storybook, tested in CI. SPF/DKIM/DMARC on the Livlux sending domain.                                                              |
| Scheduled work     | Vercel Cron → authenticated Route Handlers, with Postgres advisory locks                                                                                       | No extra infrastructure. Locks prevent overlapping runs on retries or red deploys.                                                                                                       |
| File storage       | Supabase Storage, private buckets, signed URLs with short TTL                                                                                                  | COIs, authority-to-book letters, PO documents, cached listing imagery.                                                                                                                   |
| PDF generation     | Stripe-hosted invoice PDFs as primary; React-PDF for Livlux-branded booking confirmations                                                                      | Do not rebuild invoicing — Stripe's invoice PDF is compliant and hosted. Brand the booking confirmation instead.                                                                         |
| Observability      | Sentry (errors + tracing), Vercel logs via a drain, PostHog (product analytics, feature flags), Better Stack or Checkly for uptime                             | See §19.                                                                                                                                                                                 |
| Testing            | Vitest (unit), Testcontainers or Supabase local (integration), Playwright (E2E), Stripe CLI + MSW for integration doubles                                      | See §20.                                                                                                                                                                                 |
| CI/CD              | GitHub Actions → Vercel preview deploys; Supabase CLI migrations gated on review                                                                               | See §21.                                                                                                                                                                                 |

**RECOMMENDATION — REGION**

Provision the Supabase project in `ap-southeast-2` (Sydney) and pin Vercel functions to `syd1`. Livlux's users, properties and data-residency expectations are Australian, and insurer procurement questionnaires routinely ask where personal information is stored. Getting this wrong is a migration, not a config change.

## 5 Environments, repository and configuration

### 5.1 Environments

| Environment | URL                        | Data                                            | Integrations                                                                                         |
|-------------|----------------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Local       | localhost:3000             | Supabase local stack (Docker), seeded fixtures  | Hostaway: recorded fixtures via MSW. Stripe: test mode + <code>stripe listen</code> .                |
| Preview     | Per-PR Vercel URL          | Shared staging Supabase project, reset weekly   | Hostaway: dedicated Hostaway <i>test account</i> with a handful of dummy listings. Stripe test mode. |
| Staging     | staging.book.livlux.com.au | Staging Supabase, anonymised copy of prod shape | Hostaway test account; Stripe test mode; emails routed to a catch-all inbox.                         |
| Production  | book.livlux.com.au         | Production Supabase, PITR enabled               | Hostaway live account (read + scoped write); Stripe live mode.                                       |

#### CRITICAL — HOSTAWAY HAS NO SANDBOX

Hostaway does not publish a sandbox environment. Create a **separate Hostaway account** (or, at minimum, a clearly-marked set of test listings that are never distributed to channels) for all non-production work, with its own Account ID and API key. Never point staging at the live Hostaway account: an accidental reservation write or calendar block propagates to Airbnb, Vrbo and Booking.com within minutes and is not cleanly reversible. Enforce this in code with a hard guard that refuses write operations unless `HOSTAWAY_ALLOW_WRITES === 'true'` and `VERCEL_ENV === 'production'`.

### 5.2 Repository layout

```

livlux-partner-portal/
├── app/
│   ├── (marketing)/           # login, invite acceptance, password reset
│   ├── (portal)/             # partner-authenticated area
│   │   ├── search/ listings/[id]/ requests/ requests/[id]/
│   │   ├── invoices/ team/ settings/
│   │   ├── (admin)/         # Livlux-authenticated area
│   │   │   ├── queue/ bookings/ partners/ rate-cards/ reports/ audit/ integrations/
│   └── api/
│       ├── quotes/ requests/ payments/ admin/ map/ cron/
│       └── webhooks/stripe/ webhooks/hostaway/
├── lib/
│   ├── domain/               # pure logic: pricing, availability, state machine, permissions
│   ├── hostaway/             # client, token store, rate limiter, mappers, types
│   ├── stripe/               # client, webhook handlers, invoice composer
│   ├── db/                   # typed queries, RLS-aware and service-role clients
│   ├── notifications/        # email templates, in-app, Slack
│   └── observability/         # logger, metrics, Sentry wiring
├── supabase/
│   ├── migrations/           # timestamped SQL, forward-only
│   └── seed/
├── tests/ {unit,integration,e2e,fixtures}/
└── docs/                     # this specification, ADRs, runbooks

```

### 5.3 Configuration principles

- All secrets live in Vercel environment variables and Supabase Vault. Nothing secret is committed, ever, including in `.env.example`.
- Only variables prefixed `NEXT_PUBLIC_` reach the browser. Audit this list in code review — a leaked service-role key is a full data breach.

- Environment variables are parsed and validated once at boot through a Zod schema in `lib/env.ts`; the app refuses to start if a required variable is missing or malformed. A missing Stripe webhook secret must fail loudly at deploy, not silently at 2am.
- Business parameters that Livlux will want to change (hold duration, SLA hours, deposit percentage, cancellation cut-off, check-in detail release offset) belong in a `settings` table with an admin UI — **not** in environment variables. See Appendix A for the boundary between the two.

## 6 Hostaway integration

Hostaway is the system of record for inventory, availability and reservations. Everything the portal shows about a property originates here. This section defines exactly how the platform reads from it, keeps in step with it, and writes back to it.

### 6.1 Authentication

Hostaway uses the OAuth 2.0 **client credentials** grant. Credentials are generated self-serve in the Hostaway dashboard under *Settings* → *Hostaway API*; the Account ID is the `client_id` and the generated API key is the `client_secret`. The key is shown once and cannot be retrieved again — store it in Vercel env vars and a password manager at the moment of creation.

```
POST https://api.hostaway.com/v1/accessTokens
Content-type: application/x-www-form-urlencoded
Cache-control: no-cache

grant_type=client_credentials
&client_id={HOSTAWAY_ACCOUNT_ID}
&client_secret={HOSTAWAY_API_KEY}
&scope=general

→ 200 { "token_type": "Bearer", "expires_in": 15897600, "access_token": "eyJ0eXAiOi..." }
```

| Property              | Behaviour and required handling                                                                                                                                                                                                                             |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Token lifetime        | Approximately 24 months ( <code>expires_in</code> ≈ 15,897,600 seconds). Do <b>not</b> request a token per call — that is both wasteful and a fast route into the rate limit.                                                                               |
| Storage               | Persist the token, its expiry and its issue time in an <code>integration_tokens</code> table (encrypted at rest via Supabase Vault or <code>pgsodium</code> ). Cache in process memory for the lifetime of the serverless instance.                         |
| Acquisition algorithm | (1) Check in-memory cache → (2) check database → (3) if absent or expiring within 30 days, mint a new token inside a Postgres advisory lock so concurrent invocations do not stampede.                                                                      |
| Post-issue delay      | Hostaway documents a caveat that a freshly issued token may not be immediately usable. Wait ≥1 second after minting before first use, and treat a 401 on the first call after minting as retryable once.                                                    |
| Revocation            | <code>DELETE https://api.hostaway.com/v1/accessTokens?token=...</code> . Wire this into an admin "rotate credentials" action so a compromised key can be cut in minutes.                                                                                    |
| 401 handling          | A 401 mid-flight means the token was revoked or the key rotated in the Hostaway dashboard. Invalidate the cache, mint once, retry once, and if it fails again raise a <b>CRITICAL</b> alert — the integration is down and availability data is going stale. |
| Expiry monitoring     | A daily job checks remaining token life and alerts at 60, 30 and 7 days. A silently expired token 20 months from now, with nobody left who remembers this system, is the single most likely long-term outage.                                               |

### 6.2 Rate limits and the request governor

Hostaway enforces roughly **15 requests per 10 seconds per source IP** and **20 requests per 10 seconds per account**, returning HTTP 429 on breach. Serverless functions do not share memory, so a naive per-instance limiter will not hold under concurrent cron runs plus live traffic.

#### Required design

- **Single choke point.** Every Hostaway call goes through `lib/hostaway/client.ts`. Direct `fetch()` to `api.hostaway.com` anywhere else is a lint-enforced error.
- **Distributed token bucket.** Implement the bucket in Postgres (a single-row counter table updated with `SELECT ... FOR UPDATE`, refilled by timestamp arithmetic) or Upstash Redis if latency demands it. Budget **12 requests / 10 seconds** — deliberately below the documented ceiling, leaving headroom for webhook-triggered refetches.

- **Priority classes.** Interactive calls (an approval-time availability check) pre-empt background sync. Implement two buckets: `interactive` (reserved 4 req/10s) and `bulk` (8 req/10s). A partner waiting on a screen must never queue behind a nightly reconciliation.
- **Retry policy.** On 429 or 5xx: exponential backoff with full jitter, base 1s, max 5 attempts, cap 30s. Honour `Retry-After` if present. Never retry non-idempotent writes without an idempotency guard (§6.7).
- **Circuit breaker.** After 5 consecutive failures, open the circuit for 60 seconds, serve cached data, and surface a banner in the admin console: *"Hostaway sync degraded — availability last confirmed at HH:MM."* Partners see a subtler version of the same truth rather than stale data presented as live.
- **Sync scheduling.** Hostaway's own guidance is to spread polling rather than concentrate it. Stagger listing syncs by a deterministic hash of listing ID across the window; never fire everything at `00:00:01`.

```
// lib/hostaway/client.ts — shape of the governed client
export async function hostawayFetch<T>({
  path: string,
  opts: { method?: string; body?: unknown; priority?: 'interactive' | 'bulk';
    idempotencyKey?: string; timeoutMs?: number } = {}
}): Promise<HostawayResult<T>> {
  await rateLimiter.acquire(opts.priority ?? 'bulk'); // blocks until budget available
  const token = await getAccessToken(); // cached, lock-guarded
  return withRetry(() => fetchWithTimeout(`${BASE}/v1${path}`, {
    method: opts.method ?? 'GET',
    headers: { Authorization: `Bearer ${token}`,
      'Content-type': 'application/json',
      'Cache-control': 'no-cache' },
    body: opts.body ? JSON.stringify(opts.body) : undefined,
  })), { retries: 5, baseMs: 1000, jitter: 'full', retryOn: [429, 500, 502, 503, 504] });
}
```

## 6.3 Endpoints consumed

| Method     | Path (base <code>https://api.hostaway.com/v1</code> )                                                                           | Use in the platform                                                                                                                                                                                                                                          |
|------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST       | <code>/accessTokens</code>                                                                                                      | Mint bearer token.                                                                                                                                                                                                                                           |
| DELETE     | <code>/accessTokens</code>                                                                                                      | Revoke on credential rotation.                                                                                                                                                                                                                               |
| GET        | <code>/listings</code>                                                                                                          | Full catalogue sync. Paginate with <code>limit / offset</code> ; prefer cursor-style <code>afterId</code> where supported to avoid gaps when the set mutates mid-pagination.                                                                                 |
| GET        | <code>/listings/{id}</code>                                                                                                     | Detail refresh; on-demand refresh from the admin console.                                                                                                                                                                                                    |
| GET        | <code>/listings/{id}/calendar?startDate=&amp;endDate=&amp;includeResources=1</code>                                             | <b>Core availability source.</b> Per-day <code>status</code> (available / blocked / reserved / pending), <code>isAvailable</code> , <code>price</code> , <code>minimumStay</code> , and — with <code>includeResources=1</code> — nested reservation context. |
| GET        | <code>/reservations</code>                                                                                                      | Reconciliation. Filter by <code>listingId</code> , <code>arrivalStartDate</code> , <code>arrivalEndDate</code> ; paginate carefully.                                                                                                                         |
| GET        | <code>/reservations/{id}</code>                                                                                                 | Authoritative re-fetch after any webhook. <b>Never trust webhook payload contents alone.</b>                                                                                                                                                                 |
| POST       | <code>/reservations</code>                                                                                                      | Create the confirmed booking on approval. Requires <code>listingMapId</code> , <code>channelId</code> (direct-booking channel), guest details, <code>arrivalDate</code> , <code>departureDate</code> , financial fields.                                     |
| PUT        | <code>/reservations/{id}</code>                                                                                                 | Modify dates/guests on an approved change; cancel by status update.                                                                                                                                                                                          |
| PUT / POST | <code>/listings/{id}/calendar</code> , <code>/listings/{id}/calendar/batch</code>                                               | <i>Optional, guarded.</i> Block dates during a confirmed hold if Livlux elects hard holds (see §6.7 and §10.4).                                                                                                                                              |
| GET        | <code>/amenities</code> , <code>/bedTypes</code> , <code>/countries</code> , <code>/currencies</code> , <code>/timezones</code> | Reference data, cached for 24h, used to resolve numeric IDs into display labels.                                                                                                                                                                             |
| GET / POST | <code>/webhooks/unifiedWebhooks</code>                                                                                          | Programmatic webhook registration and verification on deploy (fall back to the dashboard if unavailable on the account).                                                                                                                                     |

| Method | Path (base <code>https://api.hostaway.com/v1</code> )        | Use in the platform                                                                                                             |
|--------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/customFields, /listings/{id}/customFieldValues</code> | Optional: source of Livlux-specific operational fields (e.g. corporate-suitability flags) if Livlux maintains them in Hostaway. |

### RESPONSE ENVELOPE

Most Hostaway endpoints wrap payloads as `{ "status": "success", "result": ... }`. Unwrap this in the client, and treat a non-`success` status as an error even when the HTTP code is 200 — this is a real failure mode that silently produces empty catalogues if ignored.

## 6.4 Synchronisation strategy

| Job                         | Frequency     | Scope                                                                  | Behaviour                                                                                                                                                                                                                                                                                                               |
|-----------------------------|---------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Listing catalogue sync      | Hourly        | All listings                                                           | Fetch all listings paginated. Compute a content hash per listing; upsert only changed rows to keep <code>updated_at</code> meaningful. Mark listings absent from the response as <code>is_active = false</code> (soft delete — never hard-delete, bookings reference them). Geocode any listing with a changed address. |
| Calendar sync — near window | Every 15 min  | Today → +120 days, all active listings                                 | The window partners actually transact in. Staggered by <code>listing_id % 15</code> so each minute handles a slice, keeping request rate flat.                                                                                                                                                                          |
| Calendar sync — far window  | Every 6 hours | +121 → +400 days                                                       | Long corporate placements book far ahead; the far window changes slowly.                                                                                                                                                                                                                                                |
| Webhook-triggered refresh   | On event      | One listing / one reservation                                          | On <code>reservation.created</code> or <code>reservation.updated</code> , re-fetch the reservation, then re-fetch the affected listing's calendar for the affected date span $\pm 1$ day.                                                                                                                               |
| Pre-approval live check     | On demand     | One listing, exact stay dates                                          | Synchronous, <code>interactive</code> priority. Runs immediately before any Hostaway reservation write. This is the guard that makes double-booking structurally very unlikely.                                                                                                                                         |
| Nightly reconciliation      | 03:15 AEST    | All listings, 400-day window; all reservations created by the platform | Full comparison of cached calendar to Hostaway; produces a drift report. Any platform-confirmed booking without a matching live Hostaway reservation raises <b>CRITICAL</b> . Any cached day disagreeing with Hostaway is corrected and counted.                                                                        |

### 6.4.1 Upsert semantics

```
-- calendar_days upsert; idempotent, safe to replay in any order
INSERT INTO calendar_days
  (listing_id, date, status, is_available, price_cents, min_stay,
   reservation_ref, source_synced_at)
VALUES ($1, $2, $3, $4, $5, $6, $7, now())
ON CONFLICT (listing_id, date) DO UPDATE SET
  status           = EXCLUDED.status,
  is_available     = EXCLUDED.is_available,
  price_cents      = EXCLUDED.price_cents,
  min_stay         = EXCLUDED.min_stay,
  reservation_ref  = EXCLUDED.reservation_ref,
  source_synced_at = EXCLUDED.source_synced_at
WHERE calendar_days.source_synced_at < EXCLUDED.source_synced_at; -- never apply stale data
```

The trailing `WHERE` clause matters: a slow bulk sync must not overwrite fresher data written seconds ago by a webhook-triggered refresh. Out-of-order arrival is normal in an event-driven system and the database should be the thing that enforces monotonicity.

## 6.4.2 Multi-unit listings

### FREQUENT SOURCE OF WRONG AVAILABILITY

For multi-unit properties, availability is governed by **allotment**, not by the day's **status** field. A day can read **reserved** while units remain free. The resolution rule is: `isAvailable = typeof allotment === 'number' ? allotment > 0 : status === 'available'`. Store **allotment** on **calendar\_days** and apply this rule in one shared helper. If Livlux's portfolio contains no multi-unit listings today, implement it anyway — it costs an hour now and is a silent correctness bug the day the first one is added.

## 6.5 Webhooks

Hostaway's **unified webhook** supports three events: **reservation created**, **reservation updated**, and **new message received**. The updated event fires on status, date, listing, pricing, payment-status, door-code and custom-field changes. Delivery is retried **3 times** on non-2xx, after which Hostaway emails a configured address and gives up.

| Concern                     | Required handling                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authenticity                | Hostaway offers HTTP Basic auth credentials on the webhook (username + password), not an HMAC signature. Configure a long random username/password pair, <i>and</i> embed an unguessable path segment: <code>/api/webhooks/hostaway/{WEBHOOK_PATH_SECRET}</code> . Compare both with constant-time equality. Reject anything else with 401 — but note that because there is no cryptographic signature, the payload must be treated as a <b>hint, not as truth</b> . |
| Never trust the payload     | On receipt, extract only the reservation ID and listing ID, then re-fetch from <code>GET /reservations/{id}</code> over the authenticated API. This neutralises both spoofing and Hostaway's own documented caveat that complex fields may lag the webhook.                                                                                                                                                                                                          |
| Fast acknowledgement        | Persist the raw body to <code>webhook_events</code> and return <b>200</b> within ~1 second. All processing happens in a separate drain. Slow handlers cause Hostaway retries, which cause duplicate processing.                                                                                                                                                                                                                                                      |
| Idempotency                 | Deduplicate on a hash of <code>(event_type, object_id, payload_digest)</code> with a unique index. A re-played delivery is recorded and skipped, not reprocessed.                                                                                                                                                                                                                                                                                                    |
| Ordering                    | No ordering guarantee exists. Every handler must be commutative with respect to arrival order — which is why all writes carry <code>source_synced_at</code> comparisons.                                                                                                                                                                                                                                                                                             |
| Missed events               | After 3 failed attempts Hostaway stops. The 15-minute calendar poll and the nightly reconciliation are the safety net; webhooks are an optimisation for latency, never the sole path. <b>Design as if webhooks will be silently lost, because occasionally they will be.</b>                                                                                                                                                                                         |
| Unknown event types         | Hostaway explicitly warns that new event types will be added. Log and 200-acknowledge unrecognised events rather than erroring — an unhandled type must not cause retry storms or failure emails.                                                                                                                                                                                                                                                                    |
| Internal-network validation | Hostaway validates the URL and rejects internal hosts. Use the public production domain; for local development use a tunnel (ngrok/Cloudflare Tunnel) pointed at a test-account webhook.                                                                                                                                                                                                                                                                             |
| Failure alert email         | Configure the webhook's recipient email to an ops-monitored address <i>and</i> ingest it — a human-only alert will be missed.                                                                                                                                                                                                                                                                                                                                        |

## 6.6 Field mapping: Hostaway → platform

Field names must be verified against live payloads. The mapping table below is the contract the rest of the platform codes against; the mapper in `lib/hostaway/mappers.ts` is the only place that knows Hostaway's shape.

### 6.6.1 Listing

| Hostaway field                             | Platform column                      | Notes                                                                                                 |
|--------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>id</code>                            | <code>hostaway_listing_id</code>     | Unique, immutable, indexed. Never the platform primary key.                                           |
| <code>listingMapId</code> / channel map id | <code>hostaway_listing_map_id</code> | <b>Required for reservation creation.</b> Distinct from <code>id</code> ; a frequent integration bug. |

| Hostaway field                                                                                                                                                                                   | Platform column                                                                                                                                                                                     | Notes                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>name</code> , <code>internalListingName</code>                                                                                                                                             | <code>public_name</code> , <code>internal_name</code>                                                                                                                                               | Internal name is shown only to Livlux staff.                                                                                                                                                   |
| <code>address</code> , <code>city</code> , <code>state</code> , <code>zipcode</code> , <code>countryCode</code>                                                                                  | <code>address_line</code> , <code>suburb</code> , <code>state</code> , <code>postcode</code> , <code>country_code</code>                                                                            | Full street address is <b>withheld from partners</b> until a booking is approved (\$14.4).                                                                                                     |
| <code>lat</code> , <code>lng</code>                                                                                                                                                              | <code>geom</code> <code>geography(Point, 4326)</code>                                                                                                                                               | Exact point stored; a jittered point is what is exposed pre-approval. Geocode from address if Hostaway returns null.                                                                           |
| <code>bedroomsNumber</code> , <code>bathroomsNumber</code> , <code>bedsNumber</code>                                                                                                             | <code>bedrooms</code> , <code>bathrooms</code> , <code>beds</code>                                                                                                                                  | Bathrooms may be fractional (2.5) — use <code>numeric(3,1)</code> , not integer.                                                                                                               |
| <code>personCapacity</code> , <code>maxChildrenAllowed</code> , <code>maxInfantsAllowed</code> , <code>maxPetsAllowed</code>                                                                     | <code>max_guests</code> , <code>max_children</code> , <code>max_infants</code> , <code>max_pets</code>                                                                                              | Drives guest-count validation at quote time.                                                                                                                                                   |
| <code>propertyTypeId</code> , <code>roomType</code> , <code>bathroomType</code>                                                                                                                  | <code>property_type</code> , <code>room_type</code> , <code>bathroom_type</code>                                                                                                                    | Resolve numeric IDs via cached reference endpoints.                                                                                                                                            |
| <code>squareMeters</code>                                                                                                                                                                        | <code>area_sqm</code>                                                                                                                                                                               | Display only; often null.                                                                                                                                                                      |
| <code>listingImages[]</code>                                                                                                                                                                     | <code>listing_images</code> ( <code>url</code> , <code>caption</code> , <code>sort_order</code> )                                                                                                   | Mirror to Supabase Storage and serve through <code>next/image</code> ; Hostaway CDN URLs can rotate and hotlinking creates a hard external dependency on page render.                          |
| <code>listingAmenities[]</code>                                                                                                                                                                  | <code>listing_amenities</code> join table                                                                                                                                                           | Map to a curated display set; surface the amenities partners filter on (Wi-Fi, parking, air-con, washer, lift, accessibility, pet-friendly, workspace).                                        |
| <code>listingBedTypes[]</code>                                                                                                                                                                   | <code>listing_bed_config</code> jsonb                                                                                                                                                               | Per-room bed configuration; a genuine decision factor for a displaced family of five.                                                                                                          |
| <code>price</code> , <code>currencyCode</code> , <code>cleaningFee</code> , <code>priceForExtraPerson</code> , <code>guestsIncluded</code> , <code>securityDepositFee</code>                     | <code>base_price_cents</code> , <code>currency</code> , <code>cleaning_fee_cents</code> , <code>extra_person_fee_cents</code> , <code>guests_included</code> , <code>security_deposit_cents</code>  | Store money as <b>integer cents</b> . Never floats.                                                                                                                                            |
| <code>weeklyDiscount</code> , <code>monthlyDiscount</code>                                                                                                                                       | <code>weekly_discount_pct</code> , <code>monthly_discount_pct</code>                                                                                                                                | Baseline long-stay discounts before any partner rate card is applied.                                                                                                                          |
| <code>minNights</code> , <code>maxNights</code>                                                                                                                                                  | <code>min_nights</code> , <code>max_nights</code>                                                                                                                                                   | Listing-level default; per-day <code>minimumStay</code> from the calendar overrides it.                                                                                                        |
| <code>checkInTimeStart</code> , <code>checkInTimeEnd</code> , <code>checkOutTime</code>                                                                                                          | <code>checkin_from</code> , <code>checkin_to</code> , <code>checkout_by</code>                                                                                                                      | Integers (hour of day) in Hostaway. Convert to time-of-day in the listing's timezone.                                                                                                          |
| <code>timeZoneName</code>                                                                                                                                                                        | <code>timezone</code>                                                                                                                                                                               | <b>Essential.</b> All check-in logic and date boundaries resolve in the property's timezone, not the server's or the user's.                                                                   |
| <code>doorSecurityCode</code>                                                                                                                                                                    | <code>access_code_encrypted</code>                                                                                                                                                                  | <b>SENSITIVE</b> Encrypted at rest, never in logs, released only per \$10.6.                                                                                                                   |
| <code>checkInInstructions</code> , <code>specialInstruction</code> , <code>doorSecurityCodeInstructions</code> , <code>houseRules</code> , <code>wifiUsername</code> , <code>wifiPassword</code> | <code>checkin_instructions</code> , <code>special_instructions</code> , <code>access_instructions</code> , <code>house_rules</code> , <code>wifi_ssid</code> , <code>wifi_password_encrypted</code> | <b>SENSITIVE</b> Same release controls as the door code.                                                                                                                                       |
| <code>description</code> , <code>thingsToNote</code> , <code>space</code> , <code>access</code> , <code>transit</code> , <code>neighborhood</code>                                               | <code>description_*</code> columns                                                                                                                                                                  | Sanitise HTML on ingest (DOMPurify server-side) — these fields frequently contain markup.                                                                                                      |
| <code>status</code> / <code>isActive</code>                                                                                                                                                      | <code>hostaway_status</code> , <code>is_active</code>                                                                                                                                               | Inactive listings disappear from search but remain resolvable for historical bookings.                                                                                                         |
| — (platform-owned)                                                                                                                                                                               | <code>is_partner_visible</code> , <code>partner_notes</code> , <code>suitability_tags</code>                                                                                                        | Livlux-controlled overlay: exclude a property from the portal without touching Hostaway; tag as <i>insurance-suitable</i> , <i>pet-friendly</i> , <i>accessible</i> , <i>corporate-ready</i> . |

## 6.6.2 Calendar day

| Hostaway field                                                | Platform column                                                   | Notes                                                                                                                                                     |
|---------------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>date</code>                                             | <code>date</code>                                                 | Property-local calendar date. Store as <code>date</code> , never <code>timestampz</code> .                                                                |
| <code>status</code>                                           | <code>status</code>                                               | Enum: <code>available</code> · <code>blocked</code> · <code>reserved</code> · <code>pending</code> . Treat unrecognised values as unavailable.            |
| <code>isAvailable</code>                                      | <code>is_available</code>                                         | Combine with <code>allotment</code> per §6.4.2.                                                                                                           |
| <code>allotment</code>                                        | <code>allotment</code>                                            | Nullable integer; multi-unit only.                                                                                                                        |
| <code>price</code>                                            | <code>price_cents</code>                                          | <b>May be null, meaning "unknown"</b> . Never substitute a default. A null price makes the day non-quotable and the listing shows "price on application". |
| <code>minimumStay</code>                                      | <code>min_stay</code>                                             | Per-day override of listing minimum.                                                                                                                      |
| <code>closedOnArrival</code> , <code>closedOnDeparture</code> | <code>closed_on_arrival</code> , <code>closed_on_departure</code> | Must be enforced in availability search or the platform will offer unbookable arrival dates.                                                              |
| <code>reservations[]</code> (includeResources=1)              | <code>reservation_ref</code>                                      | Store the ID and channel only. <b>Do not store other guests' personal data</b> — it is not needed and creates privacy exposure.                           |

## 6.7 Writing back to Hostaway

This is the highest-risk code in the system: a bad write propagates to every connected OTA channel.

| Operation                                                         | Trigger                                               | Safeguards                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Create reservation</b><br><code>POST /reservations</code>      | Livlux approval + successful payment authorisation    | (1) Live availability re-check immediately prior. (2) Idempotency key stored on the booking row <i>before</i> the call; if the row already carries a <code>hostaway_reservation_id</code> , skip. (3) Wrapped in an outbox pattern: record intent → call → record outcome, so a timeout never leaves ambiguity. (4) On timeout, do not retry blindly — query <code>GET /reservations</code> filtered by listing and dates to determine whether the write actually landed. |
| <b>Update reservation</b><br><code>PUT /reservations/{id}</code>  | Approved extension, date change or guest-count change | Re-check availability for newly-added nights only. Never shorten a stay without an explicit approved modification record.                                                                                                                                                                                                                                                                                                                                                 |
| <b>Cancel reservation</b>                                         | Approved cancellation                                 | Status update in Hostaway, then refund/void in Stripe — in that order, so inventory is released even if the refund needs manual follow-up.                                                                                                                                                                                                                                                                                                                                |
| <b>Block calendar (hard hold)</b><br><code>/calendar/batch</code> | <i>Optional</i> — only if Livlux chooses hard holds   | Hard holds remove inventory from all OTA channels for the hold duration. Default is <b>off</b> : holds are soft, in the platform database only. See §10.4 for the trade-off.                                                                                                                                                                                                                                                                                              |

```
// Outbox pattern — the only safe way to write to an external system from a transaction
// 1. In one DB transaction: mark intent
await db.tx(async t => {
  await t.bookingRequests.update(id, { status: 'CONFIRMING',
    hostaway_write_key: idemKey, hostaway_write_attempted_at: new Date() });
  await t.auditLog.append({ action: 'hostaway.reservation.create.attempt', bookingId: id });
});
// 2. Outside the transaction: call Hostaway
const res = await createHostawayReservation(payload, idemKey);
// 3. Record outcome; on ANY ambiguity, reconcile by query rather than retry
```

**DIRECT-BOOKING CHANNEL**

Reservations created via the API must be attributed to the correct `channelId` (the direct / partner channel) so that Livlux's reporting, commission calculations and owner statements do not misattribute partner revenue to an OTA. Confirm the correct channel ID with Hostaway support for the Livlux account and store it in settings, not hard-coded. Consider also writing the platform booking reference into a Hostaway custom field so the two systems are mutually traceable from either side.

## 7 Data model and database schema

### 7.1 Entity relationships

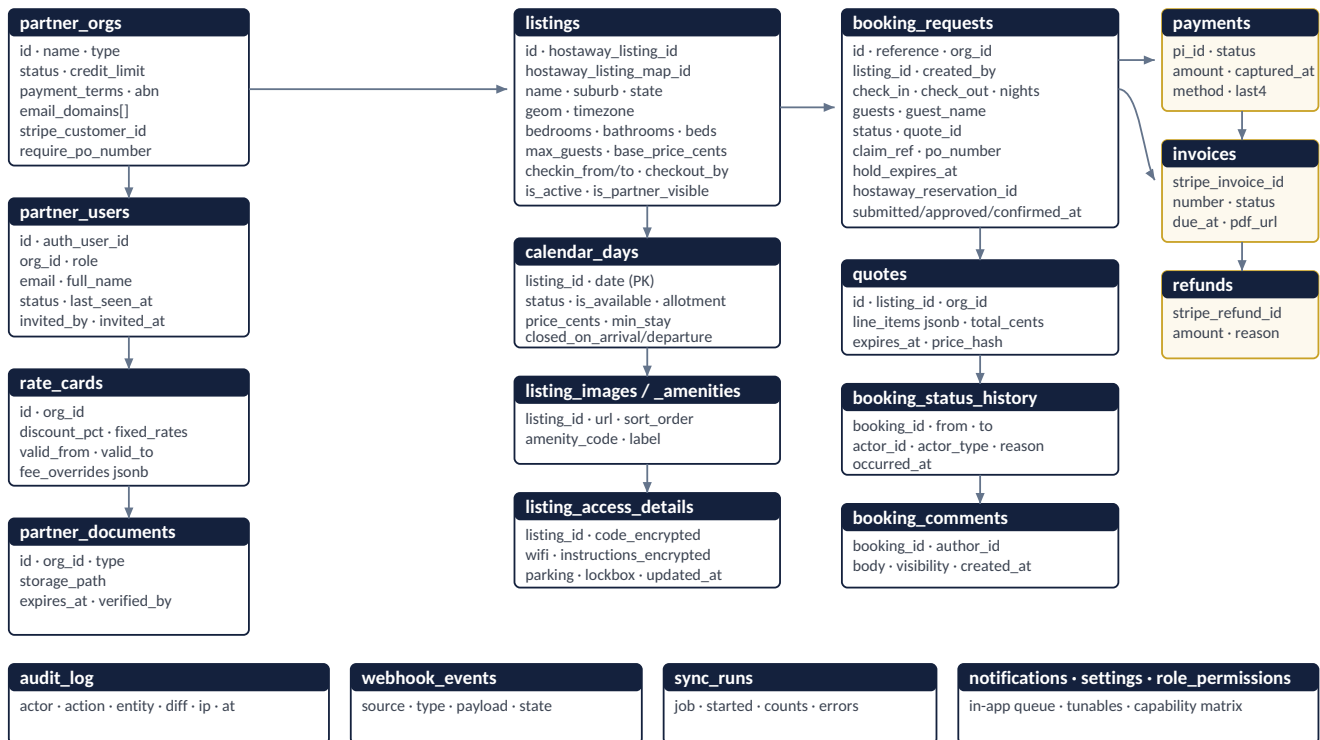


Figure 3 — Entity relationship overview (gold = entities mirroring Stripe objects)

### 7.2 Core schema (abridged DDL)

The following is the shape to implement, not a copy-paste artefact — indexes, constraints and comments should be expanded during build. All monetary values are `bigint` cents. All timestamps are `timestamptz` stored in UTC. All primary keys are `uuid` except `calendar_days`, which is keyed naturally.

```

create type partner_type      as enum ('insurance','corporate','government','other');
create type org_status       as enum ('active','suspended','pending','closed');
create type user_role        as enum ('partner_admin','partner_booker','partner_viewer','partner_finance',
                                     'livlux_ops','livlux_finance','livlux_admin');
create type booking_status   as enum ('DRAFT','SUBMITTED','UNDER_REVIEW','COUNTER_OFFERED','APPROVED',
                                     'AWAITING_PAYMENT','CONFIRMED','CHECKED_IN','COMPLETED',
                                     'DECLINED','EXPIRED','CANCELLED','FAILED');
create type day_status       as enum ('available','blocked','reserved','pending','unknown');

create table partner_orgs (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  type        partner_type not null,
  status      org_status not null default 'pending',
  abn         text,
  email_domains text[] not null default '{}',    -- auto-assign invited users
  stripe_customer_id text unique,
  payment_terms text not null default 'prepay', -- prepay | net14 | net30
  credit_limit_cents bigint,
  require_po_number boolean not null default false,
  require_claim_ref boolean not null default false,
  auto_approve_under_cents bigint,              -- optional fast path
  created_at  timestampz not null default now(),
  updated_at  timestampz not null default now()
);

create table partner_users (
  id          uuid primary key default gen_random_uuid(),
  auth_user_id uuid not null unique references auth.users(id) on delete cascade,
  org_id      uuid not null references partner_orgs(id),
  role        user_role not null,
  email       citext not null,
  full_name   text,
  phone       text,
  status      text not null default 'invited',   -- invited | active | disabled
  invited_by  uuid references partner_users(id),
  invited_at  timestampz,
  accepted_at timestampz,
  last_seen_at timestampz,
  created_at  timestampz not null default now()
);
create unique index on partner_users (org_id, email);

create table listings (
  id          uuid primary key default gen_random_uuid(),
  hostaway_listing_id bigint not null unique,
  hostaway_listing_map_id bigint,
  public_name text not null,
  internal_name text,
  description_html text,
  address_line text,
  suburb       text,
  state        text,
  postcode     text,
  country_code text default 'AU',
  geom         geography(Point,4326),
  geom_public  geography(Point,4326),          -- jittered, partner-visible pre-approval
  timezone     text not null default 'Australia/Melbourne',
  bedrooms     smallint,
  bathrooms    numeric(3,1),
  beds         smallint,
  bed_config   jsonb,
  max_guests   smallint,
  max_pets     smallint default 0,
  property_type text,
  area_sqm     numeric(7,1),
  base_price_cents bigint,
  currency     char(3) not null default 'AUD',
  cleaning_fee_cents bigint default 0,
  extra_person_fee_cents bigint default 0,
  guests_included smallint default 1,
  security_deposit_cents bigint default 0,
  weekly_discount_pct numeric(5,2) default 0,

```

```

monthly_discount_pct    numeric(5,2) default 0,
min_nights              smallint default 1,
max_nights              smallint,
checkin_from           time,
checkin_to             time,
checkout_by            time,
hostaway_status        text,
is_active              boolean not null default true,
is_partner_visible     boolean not null default true,
suitability_tags       text[] default '{}',
content_hash           text,
last_synced_at         timestampz,
created_at             timestampz not null default now(),
updated_at             timestampz not null default now()
);
create index on listings using gist (geom);
create index on listings (is_active, is_partner_visible);
create index on listings (suburb, state);

create table calendar_days (
  listing_id            uuid not null references listings(id) on delete cascade,
  date                 date not null,
  status              day_status not null default 'unknown',
  is_available        boolean not null default false,
  allotment           integer,
  price_cents         bigint,                -- NULL = unknown, never default
  min_stay            smallint,
  closed_on_arrival   boolean not null default false,
  closed_on_departure boolean not null default false,
  reservation_ref     text,
  source_synced_at    timestampz not null default now(),
  primary key (listing_id, date)
);
create index on calendar_days (date) where is_available;
create index on calendar_days (listing_id, date) include (price_cents, min_stay);

create table booking_requests (
  id                   uuid primary key default gen_random_uuid(),
  reference            text not null unique,      -- e.g. LLX-2026-004182
  org_id              uuid not null references partner_orgs(id),
  listing_id          uuid not null references listings(id),
  quote_id            uuid references quotes(id),
  created_by          uuid not null references partner_users(id),
  on_behalf_of        boolean not null default false,
  status              booking_status not null default 'DRAFT',
  check_in            date not null,
  check_out           date not null,
  nights              integer generated always as (check_out - check_in) stored,
  guests_adults       smallint not null default 1,
  guests_children     smallint not null default 0,
  guests_pets         smallint not null default 0,
  guest_name          text,
  guest_email         citext,
  guest_phone         text,
  claim_ref           text,
  po_number           text,
  cost_centre         text,
  notes               text,
  total_cents         bigint,
  currency            char(3) not null default 'AUD',
  hold_expires_at     timestampz,
  decision_due_at     timestampz,
  hostaway_reservation_id bigint,
  hostaway_write_key  text,
  checkin_released_at timestampz,
  submitted_at        timestampz,
  approved_at         timestampz,
  approved_by         uuid references partner_users(id),
  confirmed_at        timestampz,
  declined_at         timestampz,
  cancelled_at        timestampz,
  created_at          timestampz not null default now(),
  updated_at          timestampz not null default now(),
  constraint chk_dates check (check_out > check_in),

```

```

constraint chk_horizon check (check_in >= current_date - 1)
);
create index on booking_requests (status, decision_due_at);
create index on booking_requests (org_id, created_at desc);
create index on booking_requests (listing_id, check_in, check_out);

```

### 7.2.1 The non-overlap constraint

The database — not the application — must make it impossible for two live holds or confirmations to overlap on the same listing. This is the structural guarantee behind the "zero double-bookings" goal.

```

create extension if not exists btree_gist;

alter table booking_requests
  add column stay_range daterange
    generated always as (daterange(check_in, check_out, '[]')) stored;

-- Only statuses that actually consume inventory participate in the constraint.
alter table booking_requests
  add constraint no_overlapping_live_bookings
  exclude using gist (
    listing_id with =,
    stay_range with &&
  ) where (status in ('APPROVED', 'AWAITING_PAYMENT', 'CONFIRMED', 'CHECKED_IN'));

```

Note the half-open range `[])`: a checkout on the 10th and a check-in on the 10th do not collide, which is correct for accommodation. Soft holds during review are handled by a parallel `inventory_holds` table with its own exclusion constraint, so that a hold blocks other partners without yet occupying the booking constraint.

```

create table inventory_holds (
  id          uuid primary key default gen_random_uuid(),
  listing_id  uuid not null references listings(id) on delete cascade,
  booking_id  uuid references booking_requests(id) on delete cascade,
  stay_range  daterange not null,
  expires_at  timestampz not null,
  created_at  timestampz not null default now(),
  exclude using gist (listing_id with =, stay_range with &&)
);
create index on inventory_holds (expires_at);

```

Expired holds are swept every 5 minutes; the sweeper deletes rather than flags, so the exclusion constraint naturally frees the dates. A partner attempting to hold dates already held receives a clean `409 DATES_HELD` rather than a constraint-violation stack trace — catch `23P01` specifically in the data layer and translate it.

## 7.3 Supporting tables

| Table                           | Purpose and key columns                                                                                                                                                                                                                                                                                                                    |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| quotes                          | Immutable priced snapshot: <code>line_items jsonb</code> , <code>subtotal_cents</code> , <code>fees_cents</code> , <code>tax_cents</code> , <code>total_cents</code> , <code>rate_card_id</code> , <code>price_hash</code> , <code>expires_at</code> . Never updated — a re-price creates a new row linked by <code>supersedes_id</code> . |
| rate_cards /<br>rate_card_rules | Per-partner commercial terms: percentage discount, fixed nightly override by property or tag, LOS tiers, fee waivers, validity window. Multiple cards per org over time; exactly one effective on a given date (enforced by an exclusion constraint on the validity range).                                                                |
| booking_status_history          | Append-only. <code>from_status</code> , <code>to_status</code> , <code>actor_id</code> , <code>actor_type</code> , <code>reason</code> , <code>metadata jsonb</code> , <code>occurred_at</code> . This is the timestamp trail the brief requires, and it is the table Livlux will rely on in any dispute.                                  |
| booking_comments                | Threaded messages per request. <code>visibility</code> ∈ { <code>shared</code> , <code>internal</code> }. Internal notes are invisible to partners at the RLS level, not merely hidden in the UI.                                                                                                                                          |

| Table                  | Purpose and key columns                                                                                                                                                                                                                                                                                                                           |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| payments               | Mirror of Stripe PaymentIntents: <code>stripe_payment_intent_id</code> , <code>status</code> , <code>amount_authorised_cents</code> , <code>amount_captured_cents</code> , <code>method_type</code> , <code>card_last4</code> , <code>authorised_at</code> , <code>captured_at</code> , <code>failure_code</code> .                               |
| invoices               | Mirror of Stripe Invoices: <code>stripe_invoice_id</code> , <code>number</code> , <code>status</code> , <code>amount_due_cents</code> , <code>due_at</code> , <code>hosted_url</code> , <code>pdf_url</code> , <code>sent_at</code> , <code>paid_at</code> .                                                                                      |
| refunds, disputes      | Stripe mirrors with reason codes and linkage to the originating booking.                                                                                                                                                                                                                                                                          |
| listing_access_details | Encrypted check-in credentials, separated from <code>listings</code> so that ordinary listing queries never touch sensitive columns and RLS can be maximally restrictive.                                                                                                                                                                         |
| partner_documents      | COIs, authority-to-book letters, PO attachments. <code>expires_at</code> drives an expiry warning job.                                                                                                                                                                                                                                            |
| notifications          | In-app notification queue: <code>user_id</code> , <code>type</code> , <code>payload</code> , <code>read_at</code> , <code>email_sent_at</code> .                                                                                                                                                                                                  |
| webhook_events         | <code>source</code> ∈ {stripe, hostaway}, <code>event_type</code> , <code>external_id</code> , <code>payload jsonb</code> , <code>state</code> ∈ {received, processing, processed, failed, skipped}, <code>attempts</code> , <code>last_error</code> . Unique index on ( <code>source</code> , <code>external_id</code> ) gives free idempotency. |
| sync_runs              | One row per scheduled job execution: job name, window, records examined/changed, API calls consumed, duration, errors. Powers the integration health dashboard.                                                                                                                                                                                   |
| audit_log              | Every privileged action: actor, action, entity type/id, before/after diff, IP, user agent, timestamp. Append-only, no UPDATE or DELETE grant to any role.                                                                                                                                                                                         |
| settings               | Key/value business tunables with typed validation and an admin UI. Changes are audited.                                                                                                                                                                                                                                                           |
| saved_searches         | Partner-defined filter sets with optional email alerts when matching inventory frees up.                                                                                                                                                                                                                                                          |

## 7.4 Retention and lifecycle

| Data                               | Retention                   | Notes                                                                                                                                                           |
|------------------------------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Booking records and status history | 7 years                     | Financial record; aligns with Australian tax record-keeping expectations.                                                                                       |
| Audit log                          | 7 years                     | Partitioned monthly; older partitions moved to cold storage.                                                                                                    |
| Guest personal details             | 24 months post-check-out    | Then minimised to initials + booking reference. Partners' end-customers are often vulnerable people; holding their details indefinitely is an unnecessary risk. |
| Check-in credentials               | Purged 7 days post-checkout | Codes rotate anyway; retaining them is pure liability.                                                                                                          |
| Webhook payloads                   | 90 days                     | Sufficient for debugging; then delete the body and keep the metadata row.                                                                                       |
| Calendar days in the past          | 13 months                   | Enough for year-on-year occupancy reporting.                                                                                                                    |

## 8 Authentication, tenancy and access control

### 8.1 Account creation is invite-only

There is no public sign-up. Every account originates from an invitation, which is what makes the portal defensible for a partner's security review.

1. **Livlux admin creates the partner organisation** with its type, commercial terms, rate card, email domain allowlist and requirements (PO number, claim reference).
2. **Livlux admin invites the first partner administrator** by email. A signed, single-use invite token is generated with a 7-day TTL and stored hashed, never in plaintext.
3. **The partner administrator accepts**, sets a password (min 12 chars, checked against the Have I Been Pwned k-anonymity range API), and optionally enrolls TOTP.
4. **That administrator invites their own colleagues** and assigns roles. Invitee email domains are checked against the org allowlist; a mismatch requires Livlux approval, which prevents a partner accidentally inviting a personal address or an outside party.
5. **Offboarding** disables the user immediately, revokes all refresh tokens, and reassigns their open requests to the partner administrator so nothing is orphaned.

### 8.2 Session and credential policy

| Control                   | Setting                                                                                                                                                                                                                         |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Access token lifetime     | 1 hour, refresh token rotation enabled, reuse detection on                                                                                                                                                                      |
| Idle timeout              | Partner 12 hours · Livlux 2 hours (admin console holds more sensitive data)                                                                                                                                                     |
| Absolute session lifetime | 30 days partner · 7 days Livlux                                                                                                                                                                                                 |
| MFA                       | <b>Required</b> for all <code>livlux_*</code> roles. Optional but strongly encouraged for <code>partner_admin</code> ; can be mandated per-org for partners that require it.                                                    |
| Password reset            | Single-use token, 1-hour TTL, invalidates all sessions on use, notifies the user by email                                                                                                                                       |
| Rate limiting             | 5 failed logins per account per 15 min → temporary lock with email notice; 20 per IP per 15 min; exponential delay on invite-token attempts                                                                                     |
| Cookies                   | <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Lax</code> ; auth state never in <code>localStorage</code>                                                                                                         |
| Impersonation             | <code>livlux_admin</code> only, requires a stated reason, time-boxed to 30 min, banner displayed throughout, every action tagged <code>impersonated_by</code> in the audit log, and the impersonated user is emailed afterwards |

### 8.3 Row-level security

RLS is the tenancy boundary. The JWT carries `org_id` and `role` as custom claims populated by a Supabase Auth hook at token issue; policies read them directly, avoiding a subquery against `partner_users` on every row.

```

create or replace function auth.org_id() returns uuid language sql stable as $$
  select nullif(current_setting('request.jwt.claims', true)::json->>'org_id','')::uuid $$;

create or replace function auth.user_role() returns text language sql stable as $$
  select current_setting('request.jwt.claims', true)::json->>'role' $$;

create or replace function auth.is_livlux() returns boolean language sql stable as $$
  select coalesce(auth.user_role() like 'livlux%', false) $$;

alter table booking_requests enable row level security;

-- Partners see only their own organisation's requests.
create policy br_select_partner on booking_requests for select
  using ( org_id = auth.org_id() or auth.is_livlux() );

-- Partners may only insert against their own org, and only in DRAFT/SUBMITTED.
create policy br_insert_partner on booking_requests for insert
  with check ( org_id = auth.org_id()
    and auth.user_role() in ('partner_admin','partner_booker')
    and status in ('DRAFT','SUBMITTED') );

-- Partners may never move a request into an approval state; only Livlux can.
create policy br_update_partner on booking_requests for update
  using ( org_id = auth.org_id() and auth.user_role() in ('partner_admin','partner_booker') )
  with check ( status in ('DRAFT','SUBMITTED','CANCELLED') );

create policy br_all_livlux on booking_requests for all
  using ( auth.is_livlux() ) with check ( auth.is_livlux() );

-- Internal comments are invisible to partners at the database level.
create policy bc_select on booking_comments for select
  using ( auth.is_livlux()
    or ( visibility = 'shared'
      and exists (select 1 from booking_requests b
        where b.id = booking_id and b.org_id = auth.org_id()) ) );

```

#### SERVICE-ROLE DISCIPLINE

The Supabase service-role key bypasses every RLS policy. It must exist in exactly one module ( `lib/db/admin.ts` ), be used only by webhook handlers, cron jobs and explicitly-audited admin operations, and never be importable from any file under `app/(portal)`. Enforce with an ESLint `no-restricted-imports` rule and a CI check. Every service-role write must still append to `audit_log` — bypassing RLS is not a licence to bypass accountability.

## 8.4 Check-in detail exposure rules

Access codes, Wi-Fi passwords and precise entry instructions are the most sensitive non-financial data in the system. Exposure is governed by a single server-side function, never by conditional rendering.

| Condition                                                              | What the partner sees                                                                                                                                 |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Browsing, no booking                                                   | Check-in window and checkout time; access <i>method</i> in general terms ("self check-in, smart lock"); no codes, no exact address.                   |
| Request submitted, not approved                                        | As above, plus approximate location (suburb + jittered map point).                                                                                    |
| Approved and paid/authorised                                           | Exact address, parking instructions, directions, building access notes. Still no door code.                                                           |
| From T-48h before check-in (configurable), or on manual release by ops | Full access pack: door/lockbox code, Wi-Fi credentials, detailed entry instructions, emergency contact. Access is logged with user, timestamp and IP. |
| After checkout + 24h                                                   | Access pack is hidden again in the UI and purged from the database after 7 days.                                                                      |
| Booking cancelled at any point                                         | Immediate revocation; if codes were already released, ops receive a task to rotate them.                                                              |

## 9 Rate, quote and pricing engine

### 9.1 Layers of price resolution

A nightly rate shown to a partner is the result of four ordered layers. Implement them as discrete, individually testable functions; the composed result is what gets frozen into a quote.

| # | Layer                     | Behaviour                                                                                                                                                                                                                             |
|---|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Base nightly rate         | Per-day <code>price_cents</code> from the Hostaway calendar. If null for any night in the range, the stay is <i>not quotable</i> — surface "rate on application" and route to a manual quote request rather than guessing.            |
| 2 | Length-of-stay adjustment | Hostaway's <code>weeklyDiscount</code> ( $\geq 7$ nights) and <code>monthlyDiscount</code> ( $\geq 28$ nights) apply as the baseline. Livlux may define additional LOS tiers in the rate card (e.g. 90+ nights).                      |
| 3 | Partner rate card         | Either a percentage adjustment off the post-LOS nightly rate, or a fixed nightly override per listing or per suitability tag. Fixed overrides win over percentages. Fee waivers (e.g. cleaning fee waived over 28 nights) apply here. |
| 4 | Fees, deposit and tax     | Cleaning fee, extra-guest fees above <code>guests_included</code> , pet fee, mid-stay clean for long bookings, security deposit (authorised or waived per partner), then GST.                                                         |

#### GST TREATMENT — CONFIRM WITH LIVLUXE'S ACCOUNTANT

In Australia, residential accommodation supplied for 28 or more continuous days is commonly treated differently for GST than short stays, and long-stay commercial residential supplies may attract a concessional treatment. This directly changes the total on exactly the bookings this platform exists to serve. **Do not encode a tax rule from this document.** Model tax as a configurable rule set in the rate engine — `tax_rules` keyed by stay length, property type and partner type — and have Livlux's accountant sign off the values before launch. Section 23 lists this as a blocking decision.

### 9.2 Worked example

Corporate partner, 35-night stay, 2 adults, 3-bedroom townhouse, nightly rate \$310 weekdays / \$360 weekends, partner rate card at 12% off, cleaning fee waived over 28 nights, mid-stay clean at \$180.

| Line                                | Calculation                         | Amount            | Notes                                                                                                     |
|-------------------------------------|-------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------|
| Base accommodation                  | $25 \times \$310 + 10 \times \$360$ | \$11,350.00       | Sum of per-night calendar prices; never an average $\times$ nights.                                       |
| Monthly LOS discount (Hostaway 10%) | - 10%                               | -\$1,135.00       | Applied because nights $\geq 28$ .                                                                        |
| Subtotal after LOS                  |                                     | \$10,215.00       |                                                                                                           |
| Partner rate card (12%)             | - 12%                               | -\$1,225.80       | Applied to post-LOS figure. Stacking order is a commercial decision — document it and keep it consistent. |
| Accommodation total                 |                                     | \$8,989.20        |                                                                                                           |
| Cleaning fee                        | Waived ( $\geq 28$ nights)          | \$0.00            | Rate card fee override.                                                                                   |
| Mid-stay clean $\times 1$           | $1 \times \$180$                    | \$180.00          | Auto-added for stays $\geq 28$ nights per Livlux policy.                                                  |
| Extra guest fee                     | 2 guests $\leq 4$ included          | \$0.00            |                                                                                                           |
| <b>Subtotal</b>                     |                                     | <b>\$9,169.20</b> |                                                                                                           |
| GST                                 | per tax rule                        | per §9.1 note     | Computed by the configured rule, shown as a separate line.                                                |
| Security deposit                    | Authorised, not captured            | \$1,000.00        | Separate Stripe authorisation or waived for contracted partners.                                          |

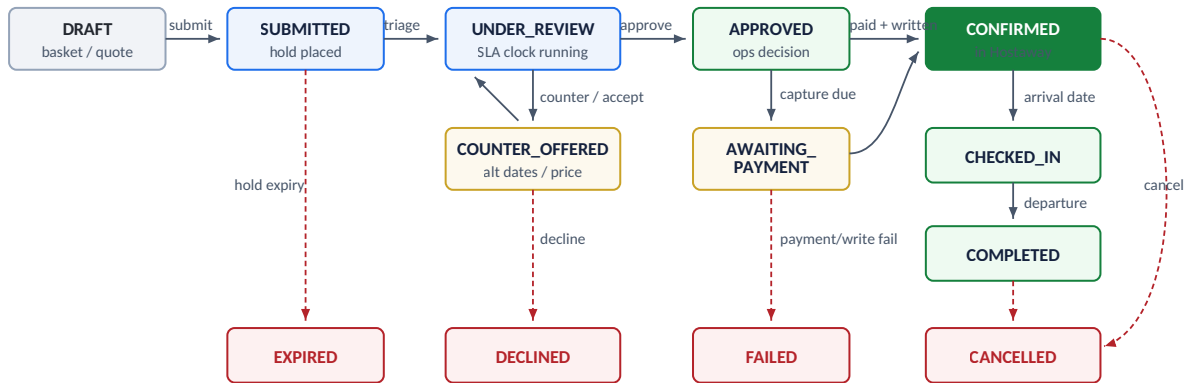
## 9.3 Quote freezing

| Rule                     | Detail                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Immutability             | A quote row is written once. Line items, inputs, rate card version and the resulting total are all stored. Re-pricing creates a new quote referencing the old one.                                                                                                                                                                                                        |
| Validity                 | Default 48 hours for partner-generated quotes (long-stay decisions need internal sign-off time), configurable per org. The quote carries <code>expires_at</code> and the UI shows a live countdown.                                                                                                                                                                       |
| Integrity hash           | <code>price_hash = sha256(listing_id · dates · guests · line_items · rate_card_id)</code> .<br>Recomputed server-side at submission and again at approval. Any mismatch aborts and forces a re-quote — this is the defence against a manipulated client payload.                                                                                                          |
| Honouring expired quotes | If the underlying price has <i>not</i> moved, silently re-issue at the same total. If it has moved by less than a configurable tolerance (default 2%), honour the original and log the absorbed variance. Beyond tolerance, present the new price for explicit acceptance. Partners will not tolerate a price changing between quote and confirmation without being told. |
| Rounding                 | All arithmetic in integer cents. Percentages applied with <code>Math.round()</code> at each discrete line, not compounded on floats. Reconcile the sum of lines to the stated total as a unit-test invariant.                                                                                                                                                             |
| Display                  | Always show nightly average <i>and</i> total; partners budget on totals but compare on nightly rates. Show the fee breakdown expanded by default — hidden fees are the fastest way to lose a corporate account.                                                                                                                                                           |

## 10 Booking request lifecycle and approval workflow

This is the core domain of the product. Every other component exists to serve this flow: a partner asks for dates, Livlux decides, money moves, Hostaway is updated, and the guest gets in the door — with an auditable timestamp at every step.

### 10.1 State machine



#### Invariants enforced at every transition

1. Only transitions in the Appendix B table are permitted; anything else raises and is logged as an integrity event, never silently ignored.
2. Every transition appends a row to `booking_status_history` with actor, actor\_type, reason and occurred\_at — inside the same transaction as the status change.
3. Entering **APPROVED** / **AWAITING\_PAYMENT** / **CONFIRMED** / **CHECKED\_IN** engages the non-overlap exclusion constraint; the DB rejects a collision regardless of application logic.
4. Money state is never inferred from a transition — it is read from Stripe. A booking is **CONFIRMED** only after both the Stripe capture/authorisation and the Hostaway write succeed.
5. Terminal states (**COMPLETED**, **DECLINED**, **EXPIRED**, **CANCELLED**) are final; reopening requires creating a new linked request, preserving the original record intact.

Figure 4 — Booking request state machine

## 10.2 End-to-end sequence

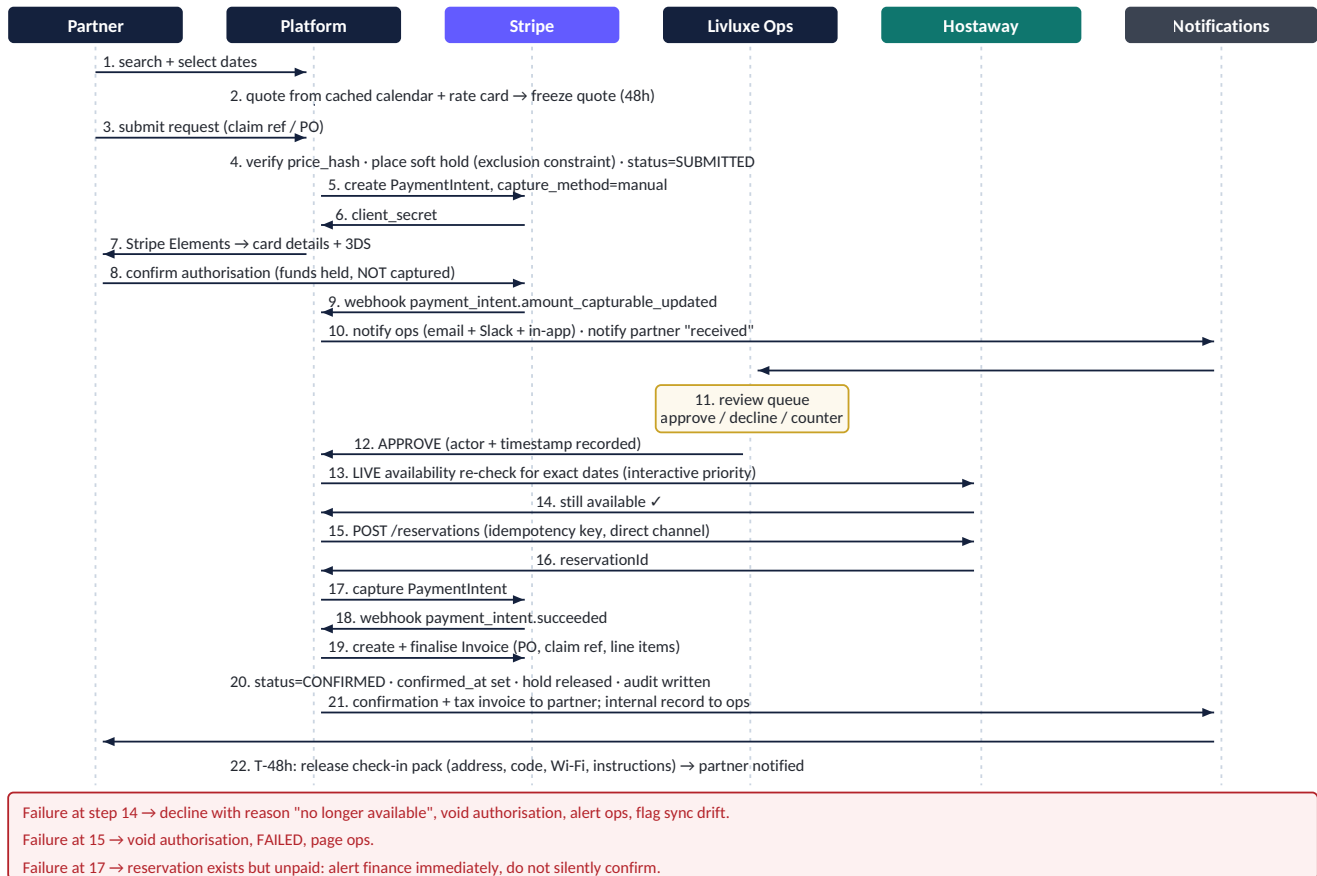


Figure 5 — Happy-path sequence with failure annotations

## 10.3 Timestamps recorded

The brief calls specifically for a timestamp on booking. In practice, a single timestamp is not enough for dispute resolution — the following are all captured, each with the acting user and their IP:

| Timestamp                              | Meaning                                                                                                                        |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| quote.created_at                       | When the price was struck and what inputs produced it.                                                                         |
| submitted_at                           | Partner pressed submit. This is the "booked at" moment from the partner's perspective and the one shown prominently in the UI. |
| payments.authorised_at                 | Funds reserved on the card.                                                                                                    |
| decision_due_at                        | SLA deadline derived from submission time and partner tier.                                                                    |
| approved_at + approved_by              | Who at Livlux approved and exactly when. Legally the most important pair in the system.                                        |
| hostaway_written_at                    | When inventory was actually committed in the PMS.                                                                              |
| payments.captured_at                   | When money moved.                                                                                                              |
| invoices.sent_at / paid_at             | Invoice lifecycle.                                                                                                             |
| confirmed_at                           | Booking is live and irrevocable without a cancellation process.                                                                |
| checkin_released_at                    | When access credentials were disclosed, and to whom.                                                                           |
| cancelled_at / declined_at<br>+ reason | With free-text reason and structured reason code.                                                                              |

All are stored UTC and rendered in the viewer's local timezone with an explicit label ("Approved 14 Sep 2026, 4:12 pm AEST by Sarah M."). Never render a bare date without a timezone in an operational system spanning states.

## 10.4 Soft holds versus hard holds

| Approach                               | Mechanism                                                                                                            | Trade-off                                                                                                                                                                                                                              |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Soft hold</b><br><b>RECOMMENDED</b> | Dates reserved only in the platform's <code>inventory_holds</code> table. Hostaway and all OTA channels remain open. | Another partner cannot take the dates, but an OTA guest still can. Risk window equals the approval SLA. Zero revenue loss from abandoned requests. Requires the live re-check at approval (§10.2 step 13) — which is mandatory anyway. |
| <b>Hard hold</b>                       | Platform blocks the dates in the Hostaway calendar for the hold duration.                                            | Guarantees the dates but removes them from every OTA channel. A partner who requests and abandons costs real revenue. Also creates a cleanup obligation: a failed hold release leaves inventory blocked and invisible.                 |

**Recommendation:** soft holds by default, with hard holds available as a per-partner or per-value setting (e.g. auto-hard-hold requests above \$15,000, or for tier-1 insurance partners with contractual response commitments). Implement both; ship with soft as the default. If hard holds are enabled, the hold-expiry sweeper must be monitored as a **CRITICAL** job — a stuck sweeper silently destroys revenue.

## 10.5 Approval, SLA and escalation

| Mechanism         | Behaviour                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Decision SLA      | Configurable per partner tier; default 4 business hours, 2 hours for tier-1 insurance partners. <code>decision_due_at</code> is computed using a business-calendar helper that understands Victorian public holidays and Livlux's operating hours.                                                                                                                                                  |
| Queue ordering    | Admin queue sorts by <code>decision_due_at</code> ascending with breach risk colour-coded. Not by creation date — urgency is not the same as age.                                                                                                                                                                                                                                                   |
| Escalation ladder | At 50% of SLA → in-app + email to assigned ops. At 80% → Slack to the ops channel. At 100% → SMS/Slack to the ops manager and the request is flagged <i>breached</i> in reporting. Escalation is a cron job, not a timer in a process.                                                                                                                                                              |
| Auto-approval     | Optional per partner: requests under <code>auto_approve_under_cents</code> , for listings tagged <i>auto-approve</i> , with no pets, no over-capacity and no date conflict, may skip manual review. Every auto-approval is logged with <code>actor_type='system'</code> and is reversible within a grace window. Ship this <b>off</b> and enable per-partner once trust in the data is established. |
| Counter-offer     | Ops propose alternative dates, an alternative property or an adjusted price. This creates a new quote and moves the request to <code>COUNTER_OFFERED</code> with its own expiry. The partner accepts (→ approval path) or declines (→ closed). Materially reduces lost bookings versus a flat decline.                                                                                              |
| Decline reasons   | Structured codes — <code>no_availability</code> , <code>unsuitable_property</code> , <code>owner_block</code> , <code>commercial_terms</code> , <code>guest_profile</code> , <code>maintenance</code> , <code>other</code> — plus free text. Structured codes make the decline report actionable; free text alone does not.                                                                         |
| Value ceiling     | <code>livlux_ops</code> may approve up to a configurable value; above it the UI forces escalation to <code>livlux_finance</code> . Enforced server-side, not just in the UI.                                                                                                                                                                                                                        |

## 10.6 Post-confirmation operations

| Operation             | Behaviour                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Check-in pack release | Cron at hourly granularity releases the access pack at <code>check_in - 48h</code> (configurable, per-property override). Ops can release early manually. Release is notified, logged and viewable in the audit trail.                                                                                                                                                                                               |
| Extension request     | Very common in insurance placements — a rebuild overruns. Partner requests additional nights from the booking screen; the system quotes only the new nights, checks availability, and on approval issues a supplementary invoice and updates the Hostaway reservation. If the property is unavailable for the extension, the system automatically surfaces the nearest suitable alternatives with a relocation note. |
| Early departure       | Partner requests; ops approve; refund calculated per the cancellation policy and processed as a partial refund against the original payment.                                                                                                                                                                                                                                                                         |
| Guest change          | Name/contact update pushed to Hostaway; requires ops approval if guest count changes materially.                                                                                                                                                                                                                                                                                                                     |

| Operation    | Behaviour                                                                                                                                                                                                                                                                                |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cancellation | Before cut-off (default 14 days out for long stays): partner self-service, automatic per-policy refund, immediate Hostaway release. After cut-off: becomes a cancellation <i>request</i> requiring ops decision, with policy-calculated retention shown to both sides before confirming. |
| Completion   | A daily job moves past-departure bookings to <b>COMPLETED</b> , triggers a partner feedback prompt, and schedules credential purge.                                                                                                                                                      |

# 11 Stripe integration, invoicing and reconciliation

## 11.1 Payment models

Partners do not all pay the same way. The platform supports three modes, selected per partner organisation.

| Mode                                               | Suits                                            | Mechanics                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A. Authorise then capture</b><br><b>DEFAULT</b> | Most corporate partners; new partners            | PaymentIntent with <code>capture_method: 'manual'</code> at submission. Funds are held, not taken. Captured on approval; voided on decline. The partner is never charged for a booking that does not happen, and Livlux never approves a booking whose card will fail.                       |
| <b>B. Deposit now, balance on invoice</b>          | High-value long stays                            | Capture a deposit (default 20%, configurable) at approval; the balance is issued as a Stripe Invoice due before check-in, with automated reminders. Reduces the working-capital objection on a \$40,000 six-month placement.                                                                 |
| <b>C. Invoice only (net terms)</b>                 | Established insurance partners with credit terms | No card. On approval, a Stripe Invoice is issued with net-14/30 terms, payable by card or BECS direct debit. Gated by <code>credit_limit_cents</code> : the system sums outstanding invoices plus the new booking and blocks (or routes to finance approval) if the limit would be exceeded. |

## 11.2 Stripe objects and configuration

| Object                | Usage                                                                                                                                                                                                                                                                                                                                |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customer              | One per <b>partner organisation</b> , not per user. Stores billing address, ABN/tax ID, saved payment methods and invoice settings. <code>metadata.livlux_org_id</code> links back.                                                                                                                                                  |
| PaymentIntent         | One per booking payment. <code>metadata</code> carries <code>booking_reference</code> , <code>org_id</code> , <code>listing_id</code> , <code>check_in</code> , <code>check_out</code> , <code>claim_ref</code> , <code>po_number</code> — this is what makes Stripe's dashboard usable by Livlux finance without cross-referencing. |
| SetupIntent           | Saves a payment method for repeat partners so subsequent bookings are one click and off-session charges are possible where authorised.                                                                                                                                                                                               |
| Invoice + InvoiceItem | Line-itemised tax invoice: accommodation, fees, discounts, tax. <code>custom_fields</code> carry PO number and claim reference. Stripe hosts the PDF and the payment page — do not rebuild either.                                                                                                                                   |
| Refund / CreditNote   | Partial or full. Credit notes for invoice-mode corrections so the tax record stays coherent.                                                                                                                                                                                                                                         |
| Dispute               | Mirrored locally; triggers an immediate finance alert and freezes any related automated refund.                                                                                                                                                                                                                                      |
| Payment methods       | Cards (with 3DS as required by SCA rules), BECS Direct Debit for AUD invoices, Link. Enable via the Payment Element rather than hard-coding a card form, so new methods need no code change.                                                                                                                                         |
| Stripe Tax            | Optional. Only enable after the GST treatment in §9.1 is settled; otherwise it will compute a rate that contradicts Livlux's accountant.                                                                                                                                                                                             |

## 11.3 Webhook events consumed

| Event                                                 | Action                                                                                                                                                   |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>payment_intent.amount_capturable_updated</code> | Authorisation succeeded. Mark payment authorised, record <code>authorised_at</code> , move request to ops queue.                                         |
| <code>payment_intent.succeeded</code>                 | Capture complete. Record <code>captured_at</code> ; if the Hostaway write has also succeeded, transition to <code>CONFIRMED</code> .                     |
| <code>payment_intent.payment_failed</code>            | Record failure code, notify partner with an actionable message, keep the hold alive for a configurable grace period so they can retry with another card. |
| <code>payment_intent.canceled</code>                  | Authorisation voided or expired. Release hold, notify, log reason.                                                                                       |

| Event                                                         | Action                                                                                                                                                                            |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>payment_intent.requires_action</code>                   | 3DS challenge outstanding. Surface it; do not treat as failure.                                                                                                                   |
| <code>charge.refunded</code>                                  | Update refund record and booking financials.                                                                                                                                      |
| <code>charge.dispute.created</code> / <code>.closed</code>    | Alert finance immediately; attach evidence pack (booking record, approval trail, invoice, check-in confirmation) — this platform's audit trail is exactly what wins a chargeback. |
| <code>invoice.finalized</code> / <code>.sent</code>           | Store hosted URL and PDF URL, record <code>sent_at</code> .                                                                                                                       |
| <code>invoice.paid</code>                                     | Mark paid; if it was the balance on a mode-B booking, clear the pre-arrival payment flag.                                                                                         |
| <code>invoice.payment_failed</code>                           | Start dunning; escalate to finance after N attempts; flag the org's credit standing.                                                                                              |
| <code>invoice.overdue</code> (or scheduled check)             | Escalation and, at threshold, suspension of new bookings for that org.                                                                                                            |
| <code>payment_method.attached</code> / <code>.detached</code> | Keep the saved-method list in sync for the partner UI.                                                                                                                            |

## Webhook handling requirements

- **Signature verification** with `stripe.webhooks.constructEvent` against the raw body. In Next.js Route Handlers read `await req.text()` — any JSON middleware that reparses the body breaks the signature.
- **Idempotency** on `event.id` via a unique index. Stripe explicitly may deliver the same event more than once.
- **Return 200 fast**, process asynchronously. Stripe retries with exponential backoff for up to ~3 days; a slow handler produces duplicates.
- **Out-of-order delivery is normal**. Before applying an update, compare against the object's current state; when in doubt, re-fetch the object from Stripe rather than trusting the event payload.
- **Never confirm a booking from a client-side redirect**. The browser can be closed, throttled or manipulated. The webhook is the only trusted signal.
- Maintain a separate endpoint secret per environment and verify the correct one is configured at boot.

## 11.4 Authorisation expiry — a scheduling constraint that shapes the product

### DESIGN CONSTRAINT

A card authorisation does not last indefinitely. Depending on card network and issuer, an uncaptured authorisation typically expires within about **7 days**, and some issuers release funds sooner. For a booking requested in March for a stay in November, holding an authorisation from submission to arrival is impossible.

Therefore: **capture at approval, not at check-in**. If Livlux wants payment closer to arrival, use mode B (small deposit captured now, balance invoiced later) or mode C (invoice with due date). Additionally, run a daily job that flags any authorisation older than 5 days and either captures, voids or escalates it — never let one silently expire, because the partner sees a pending charge disappear and assumes the booking is cancelled.

## 11.5 Invoice composition

| Element       | Content                                                                                                                                                                                                |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Header        | Livlux entity name, ABN, address, contact; invoice number (Stripe-generated, sequential); issue and due dates.                                                                                         |
| Bill-to       | Partner org legal name, ABN, billing address, billing contact, and the booking user's name.                                                                                                            |
| Custom fields | Booking reference, claim reference, PO number, cost centre, property name, stay dates, guest name. These are what let the partner's AP team match and pay without calling.                             |
| Line items    | Accommodation (nights × effective nightly rate), LOS discount, partner discount, cleaning, mid-stay clean, extra guest, pet fee, each as its own line. GST as a separate line per the configured rule. |
| Footer        | Payment methods, remittance details, terms, cancellation policy summary, and a link to the booking in the portal.                                                                                      |
| Delivery      | Emailed to the org's billing contact <i>and</i> the booking user, plus available for download in the portal. Stripe's hosted PDF is the canonical artefact.                                            |

| Element       | Content                                                                                                                                                                                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Consolidation | Optional per partner: instead of one invoice per booking, a monthly consolidated invoice covering all bookings confirmed in the period. Large insurers usually prefer this. Implement as a scheduled invoice with pending invoice items accumulated over the month. |

## 11.6 Reconciliation

A nightly job proves that the platform's view of money matches Stripe's, and raises variances rather than waiting for someone to notice at month end.

- Pull Stripe balance transactions for the period; match each to a local `payments` or `invoices` row via metadata.
- Assert: every `CONFIRMED` booking has a captured payment or an issued invoice; every captured payment has a confirmed booking; every refund has an approving actor.
- Report: gross, fees, net, refunds, disputes, outstanding authorisations, overdue invoices, and any unmatched object on either side.
- Export CSV/Xero-compatible output for Livlux finance.
- Any unmatched item older than 24 hours raises a **HIGH** alert. Reconciliation exceptions do not age quietly.

## 12 Notifications, messaging and comment threads

### 12.1 Channels

| Channel                    | Use                                                                                                                          | Notes                                                                                                                                                   |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transactional email        | Every externally-meaningful event: submission, approval, decline, counter-offer, payment, invoice, check-in pack, reminders. | Resend + React Email. SPF/DKIM/DMARC configured on the sending subdomain. Transactional mail has no unsubscribe; operational notices are not marketing. |
| In-app notification centre | Bell icon with unread count; full history; deep links to the request.                                                        | Backed by the <code>notifications</code> table; Supabase Realtime pushes live updates so the ops queue moves without a refresh.                         |
| Slack (Livlux only)        | New request, SLA breach risk, payment failure, sync degradation, double-booking detection.                                   | Incoming webhook to an <code>#bookings</code> channel; alerts route to <code>#alerts</code> .                                                           |
| SMS (optional)             | SLA breach to on-call ops; urgent same-day requests outside business hours.                                                  | Twilio. Keep narrow — SMS fatigue defeats the purpose.                                                                                                  |
| Digest email               | Daily ops summary; weekly partner summary of active placements and upcoming checkouts.                                       | Reduces the need to check the portal daily.                                                                                                             |

### 12.2 Notification matrix

| Event                          | Partner recipients                                  | Livlux recipients                            |
|--------------------------------|-----------------------------------------------------|----------------------------------------------|
| Request submitted              | Submitter (confirmation of receipt, SLA stated)     | Ops queue: email + Slack + in-app            |
| Payment authorised             | Submitter                                           | —                                            |
| Payment failed                 | Submitter + org admin                               | Ops (if the request is already in the queue) |
| <b>Approved</b>                | Submitter + org admin + billing contact             | Approver (record copy), finance              |
| Declined / counter-offered     | Submitter + org admin, with reason and alternatives | Ops record copy                              |
| Invoice issued                 | Billing contact + submitter                         | Finance                                      |
| Invoice overdue                | Billing contact, escalating to org admin            | Finance at threshold                         |
| Check-in pack released         | Submitter + nominated on-site contact               | —                                            |
| Comment added                  | Other party on the thread                           | Assigned ops                                 |
| SLA at 80% / breached          | —                                                   | Ops → ops manager                            |
| Hold expiring in 2h            | Submitter                                           | Ops                                          |
| Upstream cancellation detected | Submitter + org admin (immediately)                 | Ops + management <b>CRITICAL</b>             |
| Stay ending in 7 days          | Submitter (prompts extension decision)              | Ops (turnover planning)                      |
| Document expiring in 30 days   | Org admin                                           | —                                            |

### 12.3 Comment threads

Each booking request carries a thread. This replaces the email chain that currently carries this conversation and, critically, keeps it attached to the record.

- **Shared** comments are visible to both sides; **internal** comments are Livlux-only and enforced by RLS, not by UI filtering.

- Attachments (photos of damage, authority letters, PO documents) upload to a private Supabase Storage bucket with virus scanning and a 25 MB per-file limit; served via short-TTL signed URLs.
- Comments are immutable after a 5-minute edit window; edits retain history. Deletion is a soft flag visible to Livlux.
- @-mentions notify specific users. Ops can mark a thread as *awaiting partner response*, which pauses the SLA clock — otherwise Livlux is penalised for a partner's delay.
- Every comment is part of the exported booking record, so a dispute can be reconstructed from a single PDF.

# 13 Frontend application specification

## 13.1 Route map

| Route                                    | Access        | Purpose                                                                                                                              |
|------------------------------------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------|
| /login, /invite/[token], /reset-password | Public        | Authentication. Branded, minimal, no public inventory exposure.                                                                      |
| /                                        | Partner       | Dashboard: active placements, pending requests, upcoming check-ins/outs, unread comments, quick search.                              |
| /search                                  | Partner       | <b>Primary screen.</b> Split map + list, filters, date range, heat map toggle.                                                       |
| /listings/[id]                           | Partner       | Full property detail, gallery, amenities, bed configuration, availability calendar, check-in info, quote builder.                    |
| /requests                                | Partner       | All org requests with status filters, search by claim/PO, CSV export.                                                                |
| /requests/[id]                           | Partner       | Single request: status timeline with timestamps, price breakdown, comments, documents, check-in pack, actions (cancel, extend, pay). |
| /basket                                  | Partner       | Multi-property request assembly and single submission.                                                                               |
| /invoices                                | Partner       | Invoice list, status, download, pay now.                                                                                             |
| /team                                    | Partner admin | Users, roles, invitations, removal.                                                                                                  |
| /settings                                | Partner       | Org profile, billing contact, default PO/cost centre, notification preferences, saved searches, documents.                           |
| /admin/queue                             | Livlux        | <b>Primary ops screen.</b> SLA-sorted request queue with inline approve/decline/counter.                                             |
| /admin/bookings, /admin/bookings/[id]    | Livlux        | All bookings across partners; full detail with internal notes and manual overrides.                                                  |
| /admin/calendar                          | Livlux        | Multi-property timeline (Gantt-style) of confirmed bookings, holds and blocks; the fastest way to spot a conflict.                   |
| /admin/partners, /admin/partners/[id]    | Livlux        | Partner orgs, users, terms, credit, rate card, documents, activity.                                                                  |
| /admin/rate-cards                        | Finance/Admin | Create and version rate cards; preview effect on a sample booking before publishing.                                                 |
| /admin/listings                          | Livlux        | Synced listings, partner-visibility toggles, suitability tags, per-property overrides, manual resync.                                |
| /admin/reports                           | Livlux        | Occupancy by partner, revenue, conversion, decline reasons, SLA performance, ADR by partner.                                         |
| /admin/integrations                      | Admin         | Hostaway sync health, token expiry, webhook log, last sync per listing, force resync, circuit-breaker status.                        |
| /admin/audit                             | Finance/Admin | Filterable audit log with export.                                                                                                    |
| /admin/settings                          | Admin         | Business tunables: hold duration, SLA hours, deposit %, cancellation cut-off, check-in release offset, tax rules.                    |

## 13.2 Key screen: /search

| Element    | Specification                                                                                                                                                                                                                                                         |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Layout     | Desktop: 55% list / 45% sticky map. Tablet: toggle. Mobile: list-first with a floating "Map" button. Map and list are bidirectionally linked — hovering a card highlights its pin; clicking a pin scrolls to the card.                                                |
| Search bar | Location (suburb/postcode autocomplete over Livlux's actual inventory, not a generic geocoder), date range, guests, bedrooms. Persisted in URL query params so a search is shareable — a claims consultant forwarding a shortlist to a colleague is a core behaviour. |

| Element               | Specification                                                                                                                                                                                                                                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Filters               | Bedrooms, bathrooms, max guests, price range (nightly and total), pets allowed, parking, accessibility, air conditioning, workspace, lift, suitability tags, min/max stay compatibility, distance from a pinned address (important for insurance — "within 10 km of the damaged home so children keep their school"). |
| Results               | Cards with hero image, name, suburb, bed/bath/guest icon row, nightly rate <i>and</i> computed total for the selected dates, availability badge, suitability tags, and a compare checkbox.                                                                                                                            |
| Sorting               | Best match, price ascending/descending, most bedrooms, closest to pinned point, recently available.                                                                                                                                                                                                                   |
| Empty/near-miss state | Never a bare "no results". Offer: nearest available dates for the same property set, same dates in adjacent suburbs, properties available for part of the range (a split stay), and a "request a manual search" action that opens a ticket with ops.                                                                  |
| Performance           | Server-rendered first page; results paginated at 24; map data fetched from a separate lightweight endpoint returning only geometry and summary fields. Target LCP < 2.0 s on a mid-range laptop over 4G.                                                                                                              |

### 13.3 Key screen: `/listings/[id]`

- **Gallery** — lightbox, keyboard navigable, lazy-loaded, captions from Hostaway.
- **At-a-glance bar** — bedrooms, bathrooms, beds, max guests, property type, floor area, pets, parking.
- **Bed configuration** — per-room breakdown; decisive for family placements.
- **Amenities** — grouped (essentials, kitchen, laundry, comfort, accessibility, outdoor, safety), with the partner's filtered amenities highlighted.
- **Availability calendar** — 3-month view, per-night pricing, minimum-stay indicators, closed-on-arrival/departure markers, existing bookings shown as unavailable without revealing who holds them. Selecting a range instantly prices it.
- **Quote panel** — sticky; full line-item breakdown; nightly average and total; expiry countdown once frozen; "Add to basket" and "Request booking" actions.
- **Check-in & access** — check-in window, checkout time, access method, parking notes, and a clear statement of what is released when (\$8.4). Displayed in the property's timezone.
- **Status strip** — live availability status, when the data was last synced from Hostaway ("Availability confirmed 3 minutes ago"), and a degraded-mode notice if sync is stale. Showing the freshness of data is what makes partners trust it.
- **Location** — pre-approval: suburb, approximate map circle, distance to amenities/transport/hospitals/schools. Post-approval: exact address and directions.
- **Internal panel** (Livlux only) — internal name, owner, cleaning turnaround, maintenance notes, past partner placements, margin at the quoted rate.

### 13.4 Key screen: `/admin/queue`

Designed for throughput. An ops user should clear twenty requests without leaving the page.

- Rows sorted by `decision_due_at`; colour bands for on-track / at-risk / breached.
- Each row expands inline to show the quote breakdown, conflicting-dates check, partner history (volume, payment record, decline rate), property turnover context and guest details.
- One-click **Approve** (with a confirmation modal showing exactly what will happen: capture \$X, write reservation Y, notify Z), **Decline** (structured reason required), **Counter-offer** (date/price/property picker), **Request info** (posts a comment and pauses SLA).
- Bulk approve for low-risk requests from trusted partners, with a per-action confirmation summary.
- Keyboard shortcuts ( `j/k` navigate, `a` approve, `d` decline ) — ops users process the same screen hundreds of times.
- Live updates via Supabase Realtime so two ops staff never work the same request; soft row-locking shows "Sarah is reviewing this".

## 13.5 Cross-cutting UI requirements

| Concern                 | Requirement                                                                                                                                                                                                                                                              |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Accessibility           | WCAG 2.2 AA. Keyboard operable throughout including the map (list fallback for all map interactions), visible focus states, 4.5:1 contrast, correct ARIA on custom widgets, respects <code>prefers-reduced-motion</code> . Insurance and government partners audit this. |
| Responsive              | 360 px to 2560 px. Field adjusters work on tablets and phones; the admin console is desktop-first but must be usable on a tablet.                                                                                                                                        |
| Loading states          | Skeletons for content, optimistic UI only where safely reversible (never on payment or approval), explicit progress on multi-step operations.                                                                                                                            |
| Error handling          | Human-readable messages with a next action and a support reference code that correlates to the server log. Never surface a raw exception or a bare status code.                                                                                                          |
| Offline / flaky network | Detect offline, queue nothing destructive, warn before submission loss, autosave drafts locally.                                                                                                                                                                         |
| Timezones               | Every date/time shows its timezone. Property dates in property-local; audit timestamps in the viewer's local with UTC on hover.                                                                                                                                          |
| Money                   | Always with currency code ( <code>AUD \$9,169.20</code> ), always the same rounding as the server, never recalculated client-side for display.                                                                                                                           |
| Branding                | Livlux palette and typography; partner-facing surfaces may carry a co-branded header with the partner's logo — a small touch that materially improves perceived legitimacy.                                                                                              |
| Print                   | Print stylesheet for the booking record and quote — partners still print for claim files.                                                                                                                                                                                |

## 14 Map and heat map specification

### 14.1 Purpose

The map answers a question the list cannot: *where* is inventory, and where is it thin? For an insurance partner placing a family near a damaged home, geography is the primary filter. The heat map layer adds a second dimension — density and intensity — so a coordinator can see at a glance that the eastern suburbs are well covered for the next month while the west is not.

### 14.2 Heat map modes

| Mode                                             | Weight per point                                                       | Reads as                                                                                                                                           |
|--------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Availability density</b> <span>DEFAULT</span> | Number of available nights within the selected date range, per listing | "Where can I actually place someone for these dates?" The most useful default for partners.                                                        |
| <b>Inventory density</b>                         | 1 per listing                                                          | Raw portfolio coverage, date-independent. Useful for corporate partners planning future placements.                                                |
| <b>Rate intensity</b>                            | Median nightly rate over the range                                     | Where the affordable and premium pockets are. Colour ramp must differ visibly from the availability ramp to avoid misreading.                      |
| <b>Occupancy pressure</b> (Livlux only)          | 1 – (available nights ÷ total nights)                                  | Operational view of which areas are saturated. Not exposed to partners — it reveals commercial position.                                           |
| <b>Demand</b> (Livlux only)                      | Count of partner requests originating in the area                      | Where partners are looking but Livlux has no stock — a direct acquisition signal, and arguably the most commercially valuable view in the product. |

### 14.3 Technical implementation

| Aspect                 | Specification                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Library                | MapLibre GL JS with vector tiles from MapTiler or Mapbox. Heat rendering via MapLibre's native <code>heatmap</code> layer type for the common case; deck.gl <code>HeatmapLayer</code> only if GPU-weighted aggregation over larger point sets becomes necessary.                                                                                                                                      |
| Data endpoint          | GET <code>/api/map/points?from=&amp;to=&amp;filters=...</code> returns GeoJSON <code>FeatureCollection</code> . Properties per feature: <code>id</code> , <code>available_nights</code> , <code>total_nights</code> , <code>median_rate_cents</code> , <code>bedrooms</code> , <code>suburb</code> , <code>is_available_full_range</code> . Deliberately minimal — no names, no images, no addresses. |
| Server aggregation     | A Postgres function computes per-listing availability counts over the requested window in a single query against <code>calendar_days</code> . Do not ship raw calendar rows to the client and aggregate in JavaScript.                                                                                                                                                                                |
| Caching                | Cache the GeoJSON response by a key of (rounded date range, filter hash) for 5 minutes with <code>stale-while-revalidate</code> ; invalidate on calendar sync via <code>revalidateTag</code> . Most partners query overlapping windows.                                                                                                                                                               |
| Clustering             | <code>supercluster</code> for the pin layer above zoom 12; the heat layer renders below zoom 13 and cross-fades to pins above it. Showing both simultaneously at high zoom is visual noise.                                                                                                                                                                                                           |
| Colour ramps           | Colour-blind safe (viridis-derived for availability; a distinct diverging ramp for rate). Never red-green. A legend with numeric anchors is mandatory — an unlabelled heat map is decoration, not information.                                                                                                                                                                                        |
| Radius & intensity     | Radius interpolated by zoom (18 px at z8 → 40 px at z13); intensity scaled so a single listing is visible but does not read as a hotspot. Tune against real inventory, not synthetic data.                                                                                                                                                                                                            |
| Performance budget     | Portfolios up to ~2,000 listings render fine as client-side GeoJSON. Above that, move to server-generated vector tiles. De-bounce map-move refetches at 300 ms; abort in-flight requests on new movement.                                                                                                                                                                                             |
| Interaction            | Hovering a hotspot shows a suburb-level tooltip (count of listings, count available, median rate). Clicking zooms and switches to pins. Pins open a compact card with a link to the listing.                                                                                                                                                                                                          |
| Accessibility fallback | A toggleable data table presenting the same aggregates by suburb, keyboard navigable and screen-reader friendly. The map is an enhancement, never the only path to the information.                                                                                                                                                                                                                   |

| Aspect         | Specification                                                                                                                                                                                  |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Draw-to-search | Polygon and radius tools ("within 10 km of this address") using PostGIS <code>ST_DWithin</code> against <code>geom</code> . This is the single most requested capability from insurance users. |

```
-- Availability aggregation powering the heat map, one query, index-backed
create or replace function map_availability(p_from date, p_to date)
returns table (listing_id uuid, lng double precision, lat double precision,
              available_nights int, total_nights int, median_rate_cents bigint)
language sql stable as $$
  select l.id,
         st_x(l.geom_public::geometry), st_y(l.geom_public::geometry),
         count(*) filter (where c.is_available)::int,
         (p_to - p_from)::int,
         percentile_cont(0.5) within group (order by c.price_cents)::bigint
  from listings l
 join calendar_days c on c.listing_id = l.id
 and c.date >= p_from and c.date < p_to
 where l.is_active and l.is_partner_visible
 group by l.id, l.geom_public;
$;
```

## 14.4 Location privacy

### RECOMMENDATION

Publish `geom_public` — the exact coordinate displaced by a deterministic random offset of 100–200 m — to all partner-facing map endpoints, and render an approximate circle rather than a precise pin until a booking is approved. The exact `geom` is used only for server-side distance queries and is revealed with the address post-approval.

This protects current guests (many of whom are vulnerable people placed after a loss), matches standard practice on every major booking platform, and prevents the portal from becoming a browsable directory of exactly where Livlux's properties are. Make the offset deterministic per listing so the pin does not visibly jitter between page loads.

# 15 Search, filtering and availability queries

## 15.1 Availability semantics

"Available" for a stay from `check_in` to `check_out` means all of the following hold. Any one failing removes the listing from results.

1. Every night from `check_in` to `check_out` - 1 exists in `calendar_days` and resolves available (applying the allotment rule of §6.4.2). A *missing* day is unavailable, never assumed available.
2. No overlapping row in `inventory_holds` or in `booking_requests` in an inventory-consuming status.
3. Stay length  $\geq$  the maximum `min_stay` across the nights in range, and  $\geq$  the listing's `min_nights`.
4. Stay length  $\leq$  `max_nights` where set.
5. `closed_on_arrival` is false on `check_in`; `closed_on_departure` is false on `check_out`.
6. Guest count  $\leq$  `max_guests`; pets  $\leq$  `max_pets`.
7. Listing is `is_active` and `is_partner_visible`.
8. Every night has a non-null `price_cents` — otherwise the listing appears in a separate "price on application" group rather than being silently dropped.

```
-- Gap-free availability. The NOT EXISTS is the correctness-critical part:
-- it fails the listing if ANY night in range is missing or unavailable.
select l.*
from listings l
where l.is_active and l.is_partner_visible
  and l.max_guests >= $guests
  and not exists (
    select 1
    from generate_series($check_in::date, $check_out::date - 1, '1 day') d(day)
    left join calendar_days c on c.listing_id = l.id and c.date = d.day
    where c.date is null
      or not (case when c.allotment is not null
                  then c.allotment > 0
                  else c.is_available end)
  )
  and not exists (
    select 1 from inventory_holds h
    where h.listing_id = l.id
      and h.stay_range && daterange($check_in, $check_out, '[]')
      and h.expires_at > now()
  )
  and not exists (
    select 1 from booking_requests b
    where b.listing_id = l.id
      and b.status in ('APPROVED', 'AWAITING_PAYMENT', 'CONFIRMED', 'CHECKED_IN')
      and b.stay_range && daterange($check_in, $check_out, '[]')
  )
  and ($check_out - $check_in) >= coalesce((
    select max(c2.min_stay) from calendar_days c2
    where c2.listing_id = l.id
      and c2.date >= $check_in and c2.date < $check_out), l.min_nights, 1);
```

## 15.2 Flexible date search

Long-stay partners are frequently flexible by a few days, and a rigid search hides viable inventory. Support:

- **$\pm N$  days** on check-in (default  $\pm 3$ ): evaluate the shifted windows and return the best-priced available option per listing, labelled with the actual dates.
- **Duration-based**: "30 nights starting any time in October" — scan for qualifying windows and return the earliest and cheapest per listing.

- **Split stays:** when no single property covers the range, propose a two-property sequence with matched handover dates. For a 90-day insurance placement this frequently rescues an otherwise lost booking.
- **Partial availability:** show properties available for part of the range in a clearly separated group, with the available sub-range stated.

## 15.3 Indexing and performance

| Index                                                                                         | Serves                                                                                                                                    |
|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>calendar_days (listing_id, date)</code><br><code>include (price_cents, min_stay)</code> | Covering index for the availability scan — the hottest query in the system.                                                               |
| <code>calendar_days (date) where is_available</code>                                          | Partial index for date-first map aggregation.                                                                                             |
| <code>listings using gist (geom)</code>                                                       | Radius and polygon search.                                                                                                                |
| <code>booking_requests using gist (listing_id, stay_range)</code>                             | Overlap checks and the exclusion constraint.                                                                                              |
| Materialised view <code>listing_availability_summary</code>                                   | Pre-aggregated next-90-day availability per listing, refreshed after each calendar sync. Serves the map and the default search instantly. |

Target: search over a 400-listing portfolio with a 90-day window returns in under 150 ms at p95. Measure with `EXPLAIN (ANALYZE, BUFFERS)` against production-scale seed data, not a ten-row fixture.

## 16 Internal API surface

### 16.1 Conventions

- JSON only. `camelCase` in transport, `snake_case` in the database, mapped at the boundary.
- Every request carries a `X-Request-Id` (generated if absent) which appears in every log line and in user-facing error messages as a support reference.
- Mutating endpoints accept `Idempotency-Key`; replays within 24 hours return the original response rather than acting twice.
- All input validated with Zod at the boundary; validation failures return field-level errors.
- Rate limits per authenticated user: 100 reads/min, 20 writes/min, 5 payment operations/min. Exceeded → 429 with `Retry-After`.

```
// Error envelope – uniform across every endpoint
{ "error": { "code": "DATES_UNAVAILABLE",
  "message": "These dates are no longer available for this property.",
  "details": { "listingId": "...", "conflictingDates": ["2026-11-02", "2026-11-03"] },
  "requestId": "req_01J8..." } }
```

### 16.2 Endpoints

| Verb         | Path                                                        | Auth          | Purpose                                                                                        |
|--------------|-------------------------------------------------------------|---------------|------------------------------------------------------------------------------------------------|
| GET          | <code>/api/listings</code>                                  | Partner       | Paginated search with filters, dates and sorting.                                              |
| GET          | <code>/api/listings/:id</code>                              | Partner       | Detail, with sensitive fields filtered by entitlement.                                         |
| GET          | <code>/api/listings/:id/calendar</code>                     | Partner       | Per-night availability, price and min-stay for a window.                                       |
| GET          | <code>/api/map/points</code>                                | Partner       | GeoJSON for pins and heat map.                                                                 |
| GET          | <code>/api/map/aggregates</code>                            | Partner       | Suburb-level rollups for the accessible table fallback.                                        |
| POST         | <code>/api/quotes</code>                                    | Partner       | Price a stay; returns a frozen quote with <code>expiresAt</code> and <code>price-Hash</code> . |
| GET          | <code>/api/quotes/:id</code>                                | Partner       | Retrieve, including expiry state.                                                              |
| POST         | <code>/api/requests</code>                                  | Partner       | Submit a booking request; validates hash, places hold, creates <code>PaymentIntent</code> .    |
| GET          | <code>/api/requests</code>                                  | Partner       | List org requests with filters.                                                                |
| GET          | <code>/api/requests/:id</code>                              | Partner       | Detail with full status timeline.                                                              |
| POST         | <code>/api/requests/:id/cancel</code>                       | Partner       | Cancel or raise a cancellation request per cut-off rules.                                      |
| POST         | <code>/api/requests/:id/extend</code>                       | Partner       | Quote and request additional nights.                                                           |
| POST         | <code>/api/requests/:id/comments</code>                     | Both          | Post a comment; <code>visibility</code> defaults to shared, internal restricted to Livlux.     |
| POST         | <code>/api/requests/:id/documents</code>                    | Both          | Attach a document (signed upload URL flow).                                                    |
| GET          | <code>/api/requests/:id/checkin</code>                      | Partner       | Check-in pack — gated by \$8.4, access logged.                                                 |
| POST         | <code>/api/payments/intent</code>                           | Partner       | Create or refresh a <code>PaymentIntent</code> for a request.                                  |
| POST         | <code>/api/payments/setup-intent</code>                     | Partner       | Save a payment method for future use.                                                          |
| GET          | <code>/api/invoices</code> , <code>/api/invoices/:id</code> | Partner       | Invoice list and detail with hosted/PDF links.                                                 |
| GET/POST     | <code>/api/team/users</code>                                | Partner admin | List and invite org users.                                                                     |
| PATCH/DELETE | <code>/api/team/users/:id</code>                            | Partner admin | Change role, disable user.                                                                     |

| Verb                   | Path                                       | Auth                | Purpose                                                                                         |
|------------------------|--------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------|
| GET                    | /api/admin/queue                           | Livlux              | SLA-ordered pending requests with enrichment.                                                   |
| POST                   | /api/admin/requests/:id/approve            | Livlux              | Approve: re-check availability → write Hostaway → capture → invoice → notify. Fully idempotent. |
| POST                   | /api/admin/requests/:id/decline            | Livlux              | Decline with structured reason; voids authorisation; releases hold.                             |
| POST                   | /api/admin/requests/:id/counter            | Livlux              | Issue a counter-offer with a new quote.                                                         |
| POST                   | /api/admin/requests/:id/re-lease-checkin   | Livlux              | Manual early release of the access pack.                                                        |
| POST                   | /api/admin/refunds                         | Finance             | Issue a full or partial refund with reason and audit entry.                                     |
| GET/<br>POST/<br>PATCH | /api/admin/partners, /api/admin/rate-cards | Admin/Finance       | Partner and commercial-terms management.                                                        |
| POST                   | /api/admin/listings/:id/resync             | Livlux              | Force an immediate Hostaway refresh for one listing.                                            |
| GET                    | /api/admin/integrations/health             | Livlux              | Token expiry, last sync per job, webhook backlog, circuit state, drift counts.                  |
| GET                    | /api/admin/reports/*                       | Livlux              | Occupancy, revenue, conversion, SLA, decline analysis; CSV export.                              |
| POST                   | /api/webhooks/stripe                       | Signature           | Stripe events.                                                                                  |
| POST                   | /api/webhooks/hostaway/:secret             | Basic + path secret | Hostaway unified webhook.                                                                       |
| POST                   | /api/cron/*                                | Cron secret         | Scheduled jobs; reject any request lacking the Vercel cron header and shared secret.            |

## 16.3 Error code catalogue

| Code                     | HTTP | Meaning and client behaviour                                                               |
|--------------------------|------|--------------------------------------------------------------------------------------------|
| DATES_UNAVAILABLE        | 409  | Nights no longer free. Client refreshes the calendar and highlights the conflict.          |
| DATES_HELD               | 409  | Another partner holds these dates. Offer a waitlist or alternatives.                       |
| QUOTE_EXPIRED            | 410  | Re-quote and show the difference explicitly.                                               |
| PRICE_CHANGED            | 409  | New total returned; requires explicit acceptance.                                          |
| PRICE_HASH_MISMATCH      | 400  | Payload tampering or stale client. Force re-quote; log as a security event.                |
| MIN_STAY_NOT_MET         | 422  | Returns the required minimum for the range.                                                |
| CAPACITY_EXCEEDED        | 422  | Guest or pet count above listing limits.                                                   |
| CREDIT_LIMIT_EXCEEDED    | 403  | Route to finance approval rather than a dead end.                                          |
| ORG_SUSPENDED            | 403  | Read-only access retained; contact details shown.                                          |
| PAYMENT_REQUIRES_ACTION  | 402  | 3DS challenge; client presents the Stripe action.                                          |
| PAYMENT_FAILED           | 402  | Decline reason mapped to plain language; prompt alternate method.                          |
| HOSTAWAY_UNAVAILABLE     | 503  | Degraded mode. Reads served from cache with a staleness notice; writes blocked.            |
| HOSTAWAY_WRITE_AMBIGUOUS | 500  | Write may have landed. Booking held in <b>CONFIRMING</b> , ops paged, reconciler resolves. |
| INVALID_STATE_TRANSITION | 409  | Action no longer valid for current status; client refetches.                               |
| RATE_LIMITED             | 429  | With <b>Retry-After</b> .                                                                  |

## 17 Edge cases and failure modes

This section is the difference between a demo and a system Livlux can put in front of a national insurer. Each row states how the condition is detected, what the system does, what the user sees, and who owns the resolution. Everything here should appear as a test case in Section 20.

### 17.1 Inventory and availability

| Scenario                                                                                    | Detection                                                                                         | System behaviour                                                                                                              | User experience                                                                                                        |
|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Two partners request the same dates simultaneously                                          | Exclusion constraint on <code>inventory_holds</code> raises <code>23P01</code>                    | First transaction to commit wins. Second is caught and translated, never surfaced as a database error.                        | Second partner sees "just taken" with alternatives and a waitlist offer, within the same interaction.                  |
| OTA guest books the same dates during the approval window                                   | Live re-check at approval (\$10.2 step 13) or webhook <code>reservation.created</code>            | Approval aborts before any write or capture. Authorisation voided. Hold released. Drift counter incremented.                  | Partner notified immediately with reason and alternatives. Ops alerted so the partner can be called, not just emailed. |
| Hostaway calendar changes after confirmation (owner block applied over a confirmed booking) | Calendar sync detects a confirmed booking's nights now unavailable with a foreign reservation ref | <b>CRITICAL</b> alert. Booking is not auto-cancelled — a human decides.                                                       | Ops paged within minutes; partner contacted by a person, never by an automated cancellation email.                     |
| Listing deactivated or deleted in Hostaway with live future bookings                        | Listing sync finds it absent or inactive                                                          | Soft-deactivate; remove from search; retain all relations. Flag affected future bookings.                                     | Ops see an actionable list "3 confirmed bookings on a deactivated property" and arrange relocation.                    |
| Calendar day missing from the API response                                                  | Gap detected in the availability query                                                            | Treated as unavailable. Gap logged; a targeted refetch is queued.                                                             | Property simply does not appear for those dates. Fail closed.                                                          |
| Nightly price is null for some nights                                                       | Quote engine encounters null                                                                      | Stay is non-quotable. Never substitute the listing base price.                                                                | "Price on application" with a one-click manual quote request to ops.                                                   |
| Minimum stay changes between quote and submission                                           | Re-validated at submission                                                                        | <code>MIN_STAY_NOT_MET</code> with the new requirement returned.                                                              | Clear message with the shortest valid stay offered as a one-click adjustment.                                          |
| Multi-unit listing with partial allotment                                                   | Allotment rule (\$6.4.2)                                                                          | Availability computed from allotment, not status.                                                                             | Correct availability; no phantom unavailability.                                                                       |
| Stay crosses a DST boundary                                                                 | Always, in spring/autumn                                                                          | Nights are counted as calendar dates in the property's timezone, never as elapsed hours. All date arithmetic uses date types. | Night count and total are always correct.                                                                              |
| Very long stay (6+ months) spanning multiple rate seasons                                   | Quote engine iterates nights                                                                      | Per-night pricing handles it naturally; the engine never extrapolates an average.                                             | Breakdown shows seasonal variation transparently.                                                                      |
| Requested check-in is today, after check-in time                                            | Validated against property-local time                                                             | Same-day booking allowed but flagged as urgent; ops notified by SMS; auto-approval disabled for same-day.                     | Warning that same-day requires confirmation of property readiness.                                                     |
| Back-to-back bookings with no cleaning gap                                                  | Turnover rule check at approval                                                                   | Warn ops; allow override with a recorded reason. Configurable minimum turnover per property.                                  | Ops sees "checkout same day — cleaning window 4 hours" before approving.                                               |

## 17.2 Payments

| Scenario                                                    | Detection                                          | System behaviour                                                                                                                                                                      | User experience                                                                                                     |
|-------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Card authorisation fails                                    | <code>payment_in-tent.payment_failed</code>        | Hold retained for a grace period (default 60 min) so the partner can retry. Request does not enter the ops queue until authorised.                                                    | Decline reason translated to plain language; prompt to try another method.                                          |
| 3DS challenge abandoned mid-flow                            | <code>requires_action</code> persists past timeout | Hold expires normally; <code>PaymentIntent</code> cancelled by the sweeper.                                                                                                           | Email with a resume link that returns to the same request.                                                          |
| Authorisation expires before approval                       | Daily job flags authorisations older than 5 days   | Ops alerted to decide before expiry; if it lapses, request moves to <code>AWAITING_PAYMENT</code> and the partner is asked to re-authorise. Never silently confirm an unpaid booking. | Partner told clearly that the hold on their card was released and why.                                              |
| Capture fails after Hostaway reservation was created        | Capture API error                                  | <b>CRITICAL</b> . Booking stays <code>AWAITING_PAYMENT</code> ; reservation retained (inventory protected); finance paged.                                                            | Ops decide: retry, switch to invoice, or cancel. Never auto-cancel a reservation a guest may already be relying on. |
| Partial refund needed (early departure)                     | Ops action                                         | Policy-calculated amount pre-filled but editable with a mandatory reason; Stripe refund created; credit note issued for invoice mode.                                                 | Partner sees the calculation and the resulting credit.                                                              |
| Chargeback raised                                           | <code>charge.dispute.created</code>                | Finance alerted; automated refunds for that booking frozen; evidence pack assembled (approval trail, invoice, comment thread, check-in access log).                                   | Internal only.                                                                                                      |
| Duplicate submission (double-click, browser retry)          | <code>Idempotency-Key</code> + unique constraint   | Second call returns the first result. Exactly one <code>PaymentIntent</code> , one hold, one request.                                                                                 | No duplicate charge, no duplicate booking, no confusing error.                                                      |
| Stripe webhook arrives before the local record is committed | Handler finds no matching booking                  | Event parked as <code>pending</code> and retried with backoff for up to 15 minutes before alerting. This race is real and common.                                                     | Invisible.                                                                                                          |
| Stripe outage                                               | API errors / status page                           | Submissions accepted and queued as <code>AWAITING_PAYMENT</code> ; holds extended; retry when service returns.                                                                        | "Payment processing is temporarily unavailable — your request is held and we'll email you to complete payment."     |
| Partner exceeds credit limit                                | Sum of outstanding invoices + new booking          | Submission blocked or routed to finance approval per configuration.                                                                                                                   | Clear message naming the finance contact — never a generic "not permitted".                                         |
| Invoice unpaid past terms                                   | Dunning schedule                                   | Escalating reminders; at threshold, new bookings for the org require finance approval; at a further threshold, org suspended (existing bookings untouched).                           | Partner admin warned well before any restriction takes effect.                                                      |
| Refund exceeds captured amount                              | Server-side validation                             | Rejected before reaching Stripe with a specific error.                                                                                                                                | Finance sees the maximum refundable amount.                                                                         |

## 17.3 Hostaway integration

| Scenario                        | Detection       | System behaviour                                                                                           | User experience                                                                           |
|---------------------------------|-----------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Access token expired or revoked | 401 on any call | Invalidate cache, mint once, retry once. Persistent failure opens the circuit and raises <b>CRITICAL</b> . | Admin banner "Hostaway connection lost". Partner-facing data marked as last-confirmed-at. |
| Rate limit (429) hit            | Response status | Backoff with jitter; bulk work deferred; interactive budget protected so user-facing checks still succeed. | Sync lags silently; no user impact if within tolerance.                                   |
| Hostaway 5xx or timeout         | Client          | Retry to a cap, then circuit-break for 60 s. Reads served from cache; <b>writes are blocked entirely</b> . | Approvals disabled with an explicit reason rather than failing mysteriously.              |

| Scenario                                                     | Detection                                                    | System behaviour                                                                                                                              | User experience                                                                      |
|--------------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Reservation write times out — outcome unknown                | No response received                                         | Do not retry blindly. Query <code>GET /reservations</code> by listing and dates to determine whether it landed; reconcile; only then proceed. | Brief "confirming your booking" state; ops alerted if unresolved in 5 minutes.       |
| Duplicate reservation created in Hostaway                    | Reconciliation finds two matching the same booking reference | <b>CRITICAL</b> alert with both IDs; guided cancellation of the duplicate; root cause logged.                                                 | Internal only, resolved before it reaches a guest.                                   |
| Webhook delivery fails 3 times and stops                     | Absence, plus Hostaway's failure email                       | 15-minute polling and nightly reconciliation close the gap. Webhook health dashboard shows time since last event per type.                    | Slightly stale data for up to 15 minutes.                                            |
| Webhook replayed or spoofed                                  | Basic auth + path secret + idempotency key                   | Duplicates recorded and skipped. Contents never trusted — the reservation is always re-fetched from the API.                                  | None.                                                                                |
| Unknown webhook event type introduced by Hostaway            | Unrecognised <code>event</code> field                        | Logged, acknowledged with 200, surfaced on the integrations dashboard for review.                                                             | None; no retry storm.                                                                |
| Listing fields change shape (API version drift)              | Zod parse failure in the mapper                              | That listing's sync fails loudly and is quarantined; other listings continue. Alert names the field.                                          | One property may go stale rather than the whole catalogue breaking.                  |
| Reservation cancelled directly in Hostaway                   | <code>reservation.updated</code> with cancelled status       | Local booking marked cancelled-upstream; refund workflow raised for finance; partner and ops notified as <b>CRITICAL</b> .                    | Partner told promptly by a person plus an email, with alternatives already attached. |
| Hostaway account credentials rotated by another staff member | 401 storm                                                    | Circuit opens; admin console shows exactly what to do and where to paste the new credentials.                                                 | Short degraded window with a clear recovery path.                                    |
| Clock skew between systems                                   | Timestamps in the future                                     | Clamp to <code>now()</code> for monotonicity comparisons; log the anomaly.                                                                    | None.                                                                                |

## 17.4 Users, partners and access

| Scenario                                                | Detection                           | System behaviour                                                                                                                                                                                                                                            | User experience                                                                |
|---------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Invite token expired or reused                          | Token validation                    | Rejected; a fresh invite can be requested with one click by the inviter.                                                                                                                                                                                    | "This invitation has expired" plus a request-new-link action.                  |
| User leaves the partner organisation with open requests | Admin disables user                 | Sessions revoked; open requests reassigned to the partner admin; notifications redirected.                                                                                                                                                                  | Nothing is orphaned; the partner admin is told what they inherited.            |
| Last partner admin removed                              | Constraint check                    | Blocked — an org must always retain at least one active admin.                                                                                                                                                                                              | Clear error explaining why.                                                    |
| Partner org suspended (unpaid invoices)                 | Finance action                      | New requests blocked; existing bookings, invoices and check-in details remain fully accessible.                                                                                                                                                             | Access to what they depend on is never removed as a collections tactic.        |
| Email domain changes after a merger                     | Admin action                        | Domain allowlist is editable; existing users unaffected.                                                                                                                                                                                                    | Handled by partner admin without contacting Livlux.                            |
| Two partners share a user (broker working for both)     | Unique constraint on auth user      | v1 constraint: one user, one org. Documented limitation; the workaround is separate accounts per email alias. Multi-org membership is a v2 schema change (a membership join table) — build the constraint so it can be relaxed without data migration pain. | Documented at onboarding.                                                      |
| Session expires mid-form                                | 401 on submit                       | Draft preserved locally; re-auth returns to the exact state.                                                                                                                                                                                                | No lost work — critical when a 6-property basket took ten minutes to assemble. |
| Partner uploads a malicious file                        | Type/size validation + malware scan | Rejected; security event logged; repeated attempts rate-limited.                                                                                                                                                                                            | Generic rejection message.                                                     |

| Scenario                                | Detection       | System behaviour                                                                                      | User experience                                     |
|-----------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Partner requests deletion of their data | Privacy request | Users anonymised; financial records retained per statutory obligation with a documented lawful basis. | Handled through a defined, auditable process (§18). |

## 17.5 Operational and platform

| Scenario                                       | Detection                 | System behaviour                                                                                                                                                   | User experience                                                                                  |
|------------------------------------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Nobody approves a request (leave, after-hours) | SLA escalation job        | Ladder escalates to manager; at 150% of SLA the request is auto-extended rather than expired, and the partner is told the truth.                                   | Partner is never left guessing; the hold is not silently dropped.                                |
| Cron job fails or does not run                 | Heartbeat monitor per job | Missed heartbeat past the threshold raises an alert naming the job. Jobs are idempotent, so a catch-up run is always safe.                                         | Data staleness bounded and visible.                                                              |
| Two cron invocations overlap                   | Advisory lock             | Second acquires no lock and exits cleanly, recording a skip.                                                                                                       | None.                                                                                            |
| Deployment mid-transaction                     | Vercel rollout            | Long operations are resumable; the outbox pattern means a half-finished approval is completed by the reconciler.                                                   | Worst case, a slightly delayed confirmation.                                                     |
| Database connection exhaustion under burst     | Pool metrics              | Supervisor/PgBouncer transaction pooling; strict per-request connection discipline; queue depth alerting.                                                          | Brief slowdown rather than errors.                                                               |
| Serverless function timeout on bulk sync       | Duration metric           | Sync is chunked and resumable by cursor, with a checkpoint in <code>sync_runs</code> ; the next invocation continues.                                              | None.                                                                                            |
| Public holiday affects SLA calculation         | Business-calendar helper  | Victorian public holidays configured and maintained; SLA excludes non-business hours.                                                                              | Ops not penalised; partners see honest deadlines.                                                |
| Partner in a different timezone (WA, NZ)       | User profile              | Property dates always property-local; UI timestamps viewer-local with explicit labels.                                                                             | No off-by-one dates.                                                                             |
| Bulk request of 20 properties                  | Basket size               | Batched hold placement in a single transaction — all or nothing. Partial success on a basket is worse than clean failure.                                          | Either the whole placement is held or none, with a clear statement of which property blocked it. |
| Search returns nothing                         | Zero results              | Fallback ladder per §13.2.                                                                                                                                         | Always a next action, never a dead end.                                                          |
| Map tile provider quota exhausted              | Tile 429/403              | Fall back to a secondary tile style or a static basemap; the list view is unaffected.                                                                              | Degraded map, working product.                                                                   |
| Extremely long stay request (12+ months)       | Validation                | Above a configurable ceiling, the request is routed to manual handling — these typically require a residential tenancy agreement rather than a short-stay booking. | "This length of stay needs a tailored agreement — we'll be in touch," with ops notified.         |

# 18 Security, privacy and compliance

## 18.1 Threat model summary

| Threat                                                | Severity | Mitigation                                                                                                                                                               |
|-------------------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cross-tenant data exposure (Partner A sees Partner B) | CRITICAL | RLS on every table; JWT claims as the tenancy key; service-role isolated to one auditable module; automated cross-tenant tests in CI that fail the build.                |
| Leak of the Supabase service-role key                 | CRITICAL | Server-only env var; lint rule preventing client import; CI secret scanning; quarterly rotation; alert on anomalous service-role query volume.                           |
| Door codes / Wi-Fi credentials exposed                | CRITICAL | Separate table, encrypted at rest, strict RLS, server-side entitlement function, access logging, automatic purge post-stay, never logged or cached.                      |
| Payment manipulation (client-altered price)           | CRITICAL | All amounts computed server-side from the frozen quote; <code>price_hash</code> verified at submission and approval; Stripe amount always derived from the server quote. |
| Forged Hostaway webhook                               | HIGH     | Basic auth + unguessable path + constant-time comparison + mandatory re-fetch from the authenticated API before acting.                                                  |
| Forged Stripe webhook                                 | HIGH     | Signature verification on the raw body; per-environment endpoint secrets.                                                                                                |
| Account takeover                                      | HIGH     | MFA for Livlux roles; breached-password checks; login rate limiting; refresh-token rotation with reuse detection; new-device email notification.                         |
| Enumeration of properties or guests                   | MEDIUM   | UUID identifiers; per-user rate limiting; no public endpoints; jittered coordinates pre-approval; no other guests' PII stored from calendar responses.                   |
| Unauthorised Hostaway writes via the platform         | HIGH     | Writes only from approval and cancellation paths; environment guard; every write audited with actor; no generic write endpoint exists.                                   |
| Insider misuse                                        | MEDIUM   | Least-privilege roles; impersonation gated, time-boxed, banner-visible and notified to the impersonated user; immutable audit log; anomaly alerting on bulk exports.     |
| XSS via synced Hostaway content                       | MEDIUM   | Server-side HTML sanitisation on ingest; strict CSP; React escaping; no <code>dangerouslySetInnerHTML</code> without sanitisation at the boundary.                       |
| Denial of service / cost amplification                | MEDIUM   | Per-user and per-IP rate limits; expensive endpoints authenticated only; map responses cached; Vercel spend limits configured.                                           |

## 18.2 Controls checklist

- TLS 1.2+ everywhere; HSTS with preload.
- Strict CSP, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy`.
- Secrets in Vercel env vars and Supabase Vault; nothing in the repository; CI secret scanning on every PR.
- Dependency scanning (Dependabot) and `npm audit` gating merges on high/critical.
- SAST in CI; manual review required on any change under `lib/db/admin.ts`, auth, or payments.
- Encryption at rest (Supabase default) plus column-level encryption for access credentials.
- PITR enabled; daily backups; **restore rehearsed quarterly** — an untested backup is not a backup.
- Structured logging with automatic PII and secret redaction.
- Annual penetration test once partner volume justifies it; remediation tracked to closure.
- Documented incident response plan with named roles and a notification decision tree.
- Vendor register: Supabase, Vercel, Stripe, Hostaway, Resend, map provider, Sentry — each with a DPA on file.

## 18.3 Privacy and regulatory

| Area    | Position                                                                                                                                                                                                             |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PCI DSS | Stripe Elements / Payment Element means card data never touches Livlux systems — SAQ-A scope. Do not build a custom card form under any circumstance; it changes the compliance obligation by an order of magnitude. |

| Area                            | Position                                                                                                                                                                                                                                                                                                           |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Australian Privacy Principles   | The platform handles personal information about displaced individuals — often people in distress. Collect the minimum (name and contact for the placement), state the purpose, store in Australia, restrict access, and delete on the schedule in §7.4. Publish a privacy policy covering the portal specifically. |
| Notifiable Data Breaches scheme | Incident response includes an assessment step and a documented decision on whether the OAIC and affected individuals must be notified, with the 30-day assessment window tracked.                                                                                                                                  |
| Data residency                  | Supabase <code>ap-southeast-2</code> ; Vercel functions pinned to <code>syd1</code> . Note honestly in the privacy policy that Stripe, Sentry and email providers may process limited data offshore.                                                                                                               |
| Insurer security questionnaires | Expect them. Maintain a standing pack: architecture diagram, data flow, encryption statement, access control model, backup/DR, breach process, vendor list, pen-test summary. Assembling this after the request delays contracts by weeks.                                                                         |
| Record keeping                  | 7-year retention on financial records; audit log immutable and exportable.                                                                                                                                                                                                                                         |
| Accessibility                   | WCAG 2.2 AA — a procurement requirement for government and large corporate partners, not merely good practice.                                                                                                                                                                                                     |

## 19 Observability, alerting and runbooks

### 19.1 What is measured

| Metric                                         | Threshold                             | Why it matters                                                                                                                           |
|------------------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Hostaway API success rate                      | < 98% over 15 min → <b>HIGH</b>       | Leading indicator of stale availability.                                                                                                 |
| Hostaway 429 rate                              | > 5% → <b>MED</b>                     | Rate governor is misconfigured or sync is too aggressive.                                                                                |
| Calendar sync lag (oldest listing)             | > 45 min → <b>HIGH</b>                | Directly determines how wrong the portal can be.                                                                                         |
| Token days remaining                           | < 30 → <b>MED</b> ; < 7 → <b>CRIT</b> | The most likely long-term outage cause.                                                                                                  |
| Webhook backlog (unprocessed)                  | > 50 or > 10 min old → <b>HIGH</b>    | Drain is stuck.                                                                                                                          |
| Reconciliation drift count                     | > 0 unexplained → <b>HIGH</b>         | Cache and source disagree.                                                                                                               |
| Confirmed booking without Hostaway reservation | Any → <b>CRIT</b>                     | The nightmare scenario: a partner believes they have a property that is not held.                                                        |
| Captured payment without confirmed booking     | Any → <b>CRIT</b>                     | Money taken without service committed.                                                                                                   |
| Uncaptured authorisations > 5 days             | Any → <b>HIGH</b>                     | About to expire silently.                                                                                                                |
| SLA breach count                               | > 2/day → <b>MED</b>                  | Ops capacity or process problem.                                                                                                         |
| Request→approval median                        | Tracked, reported weekly              | The headline product metric.                                                                                                             |
| Search p95 latency                             | > 800 ms → <b>MED</b>                 | Partner-perceived quality.                                                                                                               |
| Error rate (5xx)                               | > 1% over 5 min → <b>HIGH</b>         | General health.                                                                                                                          |
| Failed logins spike                            | Statistical anomaly → <b>MED</b>      | Credential stuffing.                                                                                                                     |
| Business metrics                               | Weekly report                         | Requests, approval rate, decline reasons, revenue by partner, occupancy contribution, ADR by partner, conversion from search to request. |

### 19.2 Logging

- Structured JSON with `requestId`, `userId`, `orgId`, `route`, `durationMs`, `outcome`.
- Every Hostaway and Stripe call logged with endpoint, duration, status and attempt number — never with bodies containing PII or secrets.
- Automatic redaction of `password`, `token`, `secret`, `doorCode`, `wifi`, card fields.
- Sentry for exceptions with release tracking, source maps and user context (ID and org only, never name or email).
- Log drain to a searchable store with 30-day retention; audit log retained separately for 7 years.

### 19.3 Runbooks

| Incident                  | First actions                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hostaway integration down | Check token expiry and circuit state on <code>/admin/integrations</code> → verify credentials in the Hostaway dashboard → check Hostaway status → confirm the portal is in degraded mode showing data freshness → disable approvals until reads are trustworthy → notify ops that manual verification is required for any confirmation.                         |
| Double booking detected   | Identify both bookings and which is older → contact the affected partner by phone within 15 minutes, not email → offer the closest comparable property at no additional cost → cancel and refund the displaced booking → root-cause via <code>booking_status_history</code> and <code>sync_runs</code> → write it up; this must be a learning event every time. |
| Webhook backlog growing   | Inspect <code>webhook_events</code> for the failing type → check the drain job heartbeat → replay failed events after fixing → verify reconciliation closes any gap.                                                                                                                                                                                            |

| Incident                                     | First actions                                                                                                                                                                                                                         |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Payment captured, Hostaway write failed      | Confirm no reservation exists → decide: retry write, or refund and decline → communicate with the partner immediately → never leave the booking in an ambiguous state overnight.                                                      |
| Upstream cancellation of a confirmed booking | Identify the guest and dates → find replacement inventory before contacting anyone → phone the partner → arrange relocation → refund or re-invoice the difference → log the root cause (owner block? channel booking? manual error?). |
| Suspected data exposure                      | Preserve logs → revoke affected sessions and rotate keys → scope the exposure from the audit log → invoke the incident response plan and the NDB assessment → notify per the decision tree.                                           |
| Cron not running                             | Check Vercel cron logs and heartbeats → invoke the job manually via the admin endpoint → verify advisory locks are not stuck from a killed process → confirm data caught up.                                                          |

## 20 Testing and quality assurance

| Layer         | Tooling                         | Coverage requirements                                                                                                                                                                                                                                                                                                                                            |
|---------------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit          | Vitest                          | Quote engine (every discount/fee/tax permutation, rounding invariants, null-price handling), availability resolver (including allotment and gap cases), state machine (every legal and illegal transition), permission evaluation, business-calendar SLA arithmetic, Hostaway mappers against recorded payloads. <b>Target ≥ 90% on <code>lib/domain</code>.</b> |
| Integration   | Vitest + local Supabase         | RLS policies (a partner user literally cannot read another org's rows), exclusion constraints under concurrent inserts, idempotent upserts with out-of-order timestamps, webhook handlers including replay and malformed payloads, outbox recovery after simulated crash.                                                                                        |
| Contract      | MSW + recorded fixtures         | Hostaway client against captured real responses, including error shapes, 429s, null prices, multi-unit allotment and the <code>status: "fail"</code> envelope. Refresh fixtures quarterly against the live test account to catch API drift.                                                                                                                      |
| Payments      | Stripe test mode + Stripe CLI   | Every scenario in §17.2 using Stripe's test cards: success, decline, insufficient funds, 3DS required, 3DS failed, dispute. Webhook replay and out-of-order delivery via <code>stripe trigger</code> .                                                                                                                                                           |
| E2E           | Playwright                      | Scenarios below, run against preview deployments on every PR and nightly against staging.                                                                                                                                                                                                                                                                        |
| Accessibility | axe-core in Playwright + manual | Zero critical violations on every route; manual keyboard and screen-reader pass on search, listing, request and checkout before release.                                                                                                                                                                                                                         |
| Load          | k6                              | Search under 50 concurrent users; 500-listing × 400-day calendar sync within the window; burst of 20 simultaneous submissions on the same property (expect exactly one hold).                                                                                                                                                                                    |
| Security      | Automated + manual              | Cross-tenant access attempts as an automated suite that must fail the build; auth bypass attempts; price tampering; webhook forgery; IDOR on every <code>:id</code> route.                                                                                                                                                                                       |

### Mandatory E2E scenarios

1. Invite → accept → set password → enrol MFA → first login.
2. Search with dates and filters → map interaction → open listing → quote → submit → authorise card → ops approve → reservation appears in Hostaway test account → capture → invoice issued → confirmation email received.
3. Submit → ops decline with reason → authorisation voided → hold released → partner notified.
4. Submit → ops counter-offer → partner accepts → completes at the new price.
5. Two browser contexts submit the same dates concurrently → exactly one succeeds, the other receives a clean conflict with alternatives.
6. Hold expires without approval → dates released → `PaymentIntent` cancelled → partner notified.
7. Quote expires → re-quote at unchanged price → honoured silently.
8. Quote expires → price moved 8% → explicit acceptance required.
9. Multi-property basket of four → single submission → all-or-nothing hold behaviour.
10. Approved booking → T-48h → check-in pack released → access logged → visible to partner.
11. Extension request → availability check → approval → supplementary invoice.
12. Cancellation before cut-off → automatic refund → Hostaway reservation cancelled.
13. Cancellation after cut-off → becomes a request → ops decision → partial refund.
14. Partner A cannot access Partner B's request by direct URL (403, and no data in the response body).
15. Internal comments invisible to partner in both the UI and the raw API response.
16. Hostaway returns 503 → approval blocked with an explicit message → recovers when service returns.
17. Hostaway reservation write times out → reconciler resolves without a duplicate.
18. Invoice-mode partner exceeds credit limit → routed to finance approval.
19. Screen-reader path through search → listing → request submission.
20. Mobile viewport: full request flow on a 390 px screen.

## UAT with real users

Before launch, run a facilitated session with at least one real claims consultant and one relocation manager, using genuine (past) placement scenarios. Watch where they hesitate. The most common failures in products like this are not bugs — they are a filter that does not match how a coordinator actually thinks about "near the damaged home", or a total that does not match what they need to quote their claimant.

## 21 Deployment, CI/CD and cutover

### 21.1 Pipeline

| Stage              | Actions                                                                                                                                                                                                                                                                                                                               |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| On pull request    | Typecheck → lint → unit + integration tests → build → Vercel preview deploy → Playwright E2E against the preview → axe accessibility scan → secret scan → dependency audit. All must pass to merge.                                                                                                                                   |
| Migrations         | Supabase CLI, timestamped and forward-only. Applied to staging automatically on merge to <code>main</code> ; to production via a manually-approved job. Every migration must be backwards-compatible with the previously deployed application version (expand → migrate → contract) so a rollback never requires a database rollback. |
| On merge to main   | Deploy to staging → smoke tests → generate and commit updated database types.                                                                                                                                                                                                                                                         |
| Production release | Manual promotion of a specific build. Vercel keeps the previous deployment for instant rollback. Release notes generated from conventional commits.                                                                                                                                                                                   |
| Post-deploy        | Automated smoke suite (login, search, quote, webhook receipt); Sentry release marker; 30-minute heightened alert sensitivity.                                                                                                                                                                                                         |

### 21.2 Scheduled jobs

| Path                           | Schedule     | Purpose                                                                |
|--------------------------------|--------------|------------------------------------------------------------------------|
| /api/cron/sync-listings        | 0 * * * *    | Full catalogue refresh.                                                |
| /api/cron/sync-calendar-near   | */15 * * * * | 0–120 day window, staggered slices.                                    |
| /api/cron/sync-calendar-far    | 0 */6 * * *  | 121–400 day window.                                                    |
| /api/cron/drain-webhooks       | * * * * *    | Process queued events with <code>SKIP LOCKED</code> .                  |
| /api/cron/expire-holds         | */5 * * * *  | Release expired holds, cancel associated <code>PaymentIntents</code> . |
| /api/cron/sla-escalation       | */10 * * * * | Escalation ladder.                                                     |
| /api/cron/release-checkin      | 0 * * * *    | Release access packs at T-48h.                                         |
| /api/cron/reconcile-hostaway   | 15 3 * * *   | Full drift comparison.                                                 |
| /api/cron/reconcile-stripe     | 45 3 * * *   | Financial reconciliation.                                              |
| /api/cron/check-authorisations | 0 9 * * *    | Flag authorisations nearing expiry.                                    |
| /api/cron/invoice-dunning      | 0 10 * * *   | Overdue reminders and escalation.                                      |
| /api/cron/complete-stays       | 30 4 * * *   | Transition past bookings, schedule credential purge.                   |
| /api/cron/token-health         | 0 8 * * *    | Token expiry warnings.                                                 |
| /api/cron/digests              | 0 7 * * *    | Daily ops and weekly partner digests.                                  |

Every job: authenticated by the Vercel cron header plus a shared secret, guarded by an advisory lock, records a `sync_runs` row, emits a heartbeat, and is safe to run twice.

### 21.3 Launch and cutover

- Pilot with one partner.** Choose a cooperative, high-volume partner. Run the portal alongside the existing email process for two weeks with deliberate reconciliation of every booking through both paths.
- Verify the Hostaway write path with real reservations** in production, on low-risk dates, before opening to partners. This is the one thing that cannot be fully proven in staging.
- Expand partner by partner.** Each onboarding includes a rate card, credit terms, user invitations and a 30-minute walkthrough. Resist a big-bang launch.

4. **Domain cutover.** Start on the Vercel URL for the pilot; move to `book.livlux.com.au` before wider rollout. Configure DNS, verify TLS, set the email sending subdomain with SPF/DKIM/DMARC, and update the Hostaway webhook URL *and verify a test delivery lands* at the new host.
5. **Two weeks of heightened monitoring** with a daily reconciliation review by a human, regardless of what the automated report says.
6. **Retire the manual process** only when the pilot partner reports the portal is faster than emailing.

## 22 Delivery plan and estimates

Estimates assume one experienced full-stack engineer working with the Hostaway test account available from day one. A second engineer compresses the calendar by roughly 35%, not 50% — the integration work does not parallelise cleanly. Figures are engineering effort only and exclude Livlux's own time on rate cards, content and partner onboarding.

| # | Phase                                            | Effort      | Deliverables and exit criteria                                                                                                                                                                                                                                                                                            |
|---|--------------------------------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 | <b>Foundations</b>                               | 1 week      | Repo, environments, Supabase projects, CI/CD, base schema and migrations, auth with invite flow, design system, layout shells. <i>Exit:</i> a user can be invited, accept, log in and see an empty dashboard in all environments.                                                                                         |
| 1 | <b>Hostaway read integration</b>                 | 2–2.5 weeks | Token management, rate governor, retry and circuit breaker, listing and calendar sync jobs, mappers with full test fixtures, reconciliation job, integration health dashboard. <i>Exit:</i> the full catalogue and 400 days of calendar are in Postgres and stay in step automatically for 72 hours without intervention. |
| 2 | <b>Search, listings, map and heat map</b>        | 2–2.5 weeks | Availability query engine, search UI with filters, listing detail, calendar component, map with clustering, heat map with all modes, radius/polygon search, accessible fallbacks. <i>Exit:</i> a partner can find a property for real dates and see an accurate indicative total.                                         |
| 3 | <b>Quoting and requests</b>                      | 2 weeks     | Rate cards, quote engine with freezing and hashing, holds with exclusion constraints, request submission, basket, comment threads, notifications. <i>Exit:</i> a request flows end-to-end to the ops queue with correct pricing and a working hold.                                                                       |
| 4 | <b>Stripe and invoicing</b>                      | 2 weeks     | All three payment modes, Payment Element, webhooks, invoicing, refunds, credit limits, dunning, reconciliation. <i>Exit:</i> every \$17.2 scenario passes in test mode.                                                                                                                                                   |
| 5 | <b>Approval workflow and Hostaway write-back</b> | 2 weeks     | Admin queue, approve/decline/counter, live re-check, outbox reservation creation, capture, SLA escalation, check-in release, extensions and cancellations. <i>Exit:</i> full happy path plus every failure mode in §17.3 verified against the Hostaway test account.                                                      |
| 6 | <b>Admin, reporting and hardening</b>            | 1.5–2 weeks | Partner management, rate card UI, reports, audit viewer, settings, accessibility remediation, load testing, security testing, runbooks. <i>Exit:</i> security and accessibility suites green; load targets met.                                                                                                           |
| 7 | <b>Pilot and launch</b>                          | 2 weeks     | Pilot partner onboarding, parallel running, UAT fixes, production Hostaway write verification, domain cutover, monitoring bed-in. <i>Exit:</i> pilot partner transacting exclusively through the portal.                                                                                                                  |

**Total: approximately 14–16 engineering weeks** to a production launch with one pilot partner, plus ongoing capacity for partner onboarding and iteration. Add 15–20% contingency; the Hostaway integration is the most likely source of overrun because its behaviour cannot be fully characterised until it is exercised against a real account.

### Sequencing advice

- **Build Phase 1 first and do not rush it.** Everything downstream depends on trustworthy data. A shortcut here surfaces as a double-booking six months later.
- **Get the Hostaway test account on day one.** If it is not available, the schedule does not start — building against guessed payload shapes wastes the time it appears to save.
- **Demo at the end of Phase 2.** The map and search are what make the product feel real to Livlux and to partners, and early feedback on filters is disproportionately valuable.
- **Treat Phase 5 as the risk phase.** Allow slack; the write-back path has the most failure modes and the highest consequences.

## 23 Risks, assumptions and open decisions

### 23.1 Risk register

| Risk                                                                          | Likelihood | Impact | Mitigation                                                                                                                                          |
|-------------------------------------------------------------------------------|------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Hostaway API behaves differently from documentation (no sandbox, known drift) | High       | High   | Test account from day one; contract tests over recorded payloads; defensive parsing with Zod; quarantine failures per listing rather than per sync. |
| Double booking reaches a guest                                                | Low        | Severe | Exclusion constraints, live re-check at approval, reconciliation, alerting, and a rehearsed runbook.                                                |
| Rate limits throttle sync as the portfolio grows                              | Medium     | Medium | Priority buckets, staggering, webhook-first updates, tightened windows; monitor calls-per-sync as a capacity metric.                                |
| GST treatment for long stays is wrong                                         | Medium     | High   | Configurable tax rules; accountant sign-off before launch; never hard-code.                                                                         |
| Partners do not adopt and revert to email                                     | Medium     | High   | Pilot with a design partner; UAT with real coordinators; make the portal genuinely faster than email for the common case; measure adoption weekly.  |
| Ops capacity becomes the bottleneck instead of the system                     | Medium     | Medium | High-throughput queue design, keyboard shortcuts, bulk actions, selective auto-approval once data is trusted.                                       |
| Card authorisation expiry conflicts with far-future bookings                  | High       | Medium | Deposit or invoice modes for long-lead bookings; daily authorisation monitoring.                                                                    |
| Hostaway changes or deprecates webhooks again                                 | Medium     | Medium | Polling is the primary correctness mechanism; webhooks are an optimisation. The system is designed to survive their loss.                           |
| Key-person dependency on the integration                                      | Medium     | Medium | This document; ADRs; runbooks; test coverage; no undocumented manual steps.                                                                         |
| Scope expansion into a public booking site mid-build                          | Medium     | Medium | Explicit out-of-scope list (\$1.5); the domain layer is reusable, so defer without waste.                                                           |

### 23.2 Decisions required from Livlux before build

| #  | Decision                                                                                                                                   | Why it blocks                                                        |
|----|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| 1  | <b>GST treatment</b> for stays of 28+ days, by property and partner type                                                                   | Changes every total in the system. Blocking for Phase 3.             |
| 2  | <b>Rate card structure</b> — percentage off, fixed nightly rates, or per-tier; and the stacking order relative to Hostaway's LOS discounts | Determines the quote engine's data model. Blocking for Phase 3.      |
| 3  | <b>Payment mode per partner</b> — who prepays, who gets net terms, what credit limits apply                                                | Blocking for Phase 4.                                                |
| 4  | <b>Soft or hard holds</b> , and the hold duration                                                                                          | Affects OTA revenue exposure. Blocking for Phase 3.                  |
| 5  | <b>Approval SLA</b> per partner tier and Livlux operating hours                                                                            | Drives escalation design and the commitments shown to partners.      |
| 6  | <b>Cancellation policy</b> for partner long stays — cut-off and retention schedule                                                         | Needed for refund automation.                                        |
| 7  | <b>Hostaway direct-booking channel ID</b> and whether a custom field should carry the platform reference                                   | Blocking for Phase 5 write-back and for correct revenue attribution. |
| 8  | <b>Which listings are partner-visible</b> , and the suitability tag vocabulary                                                             | Needed for Phase 2 filtering to be meaningful.                       |
| 9  | <b>Check-in detail release timing</b> — T-48h default, or per-property                                                                     | Security-relevant default.                                           |
| 10 | <b>Hostaway test account</b> provisioning                                                                                                  | Blocking for Phase 1. Highest-priority item on this list.            |

| #  | Decision                                             | Why it blocks                                                          |
|----|------------------------------------------------------|------------------------------------------------------------------------|
| 11 | <b>Livlux legal entity, ABN and invoice branding</b> | Blocking for Phase 4 invoicing.                                        |
| 12 | <b>Pilot partner selection</b>                       | Shapes Phase 7 and, ideally, informs UI decisions from Phase 2 onward. |

## 23.3 Assumptions

- Livlux holds a Hostaway plan with Public API access and can create API credentials and unified webhooks.
- Portfolio is in the low hundreds of listings; the architecture holds comfortably to roughly 2,000 without redesign.
- Partner count is in the tens, with hundreds of users — not thousands.
- All transactions are AUD.
- Livlux has, or will establish, a Stripe account capable of the required payment methods.
- Guest identity and suitability remain the partner's contractual responsibility.
- Properties remain managed in Hostaway; the portal never becomes the primary PMS.

## Appendix A Environment variables

| Variable                                               | Scope  | Notes                                               |
|--------------------------------------------------------|--------|-----------------------------------------------------|
| NEXT_PUBLIC_SUPABASE_URL                               | Client | Project URL.                                        |
| NEXT_PUBLIC_SUPABASE_ANON_KEY                          | Client | Safe only because RLS is correct.                   |
| SUPABASE_SERVICE_ROLE_KEY                              | Server | <b>SECRET</b> Bypasses RLS. One module only.        |
| SUPABASE_DB_URL                                        | Server | Direct connection for migrations and jobs.          |
| HOSTAWAY_ACCOUNT_ID                                    | Server | OAuth <code>client_id</code> .                      |
| HOSTAWAY_API_KEY                                       | Server | <b>SECRET</b> Shown once at creation.               |
| HOSTAWAY_WEBHOOK_USER /<br>_PASSWORD                   | Server | <b>SECRET</b> Basic auth pair.                      |
| HOSTAWAY_WEBHOOK_PATH_SECRET                           | Server | <b>SECRET</b> Unguessable path segment.             |
| HOSTAWAY_ALLOW_WRITES                                  | Server | Hard guard; <code>'true'</code> only in production. |
| HOSTAWAY_DIRECT_CHANNEL_ID                             | Server | Channel attribution for created reservations.       |
| STRIPE_SECRET_KEY                                      | Server | <b>SECRET</b>                                       |
| NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY                     | Client | Payment Element.                                    |
| STRIPE_WEBHOOK_SECRET                                  | Server | <b>SECRET</b> Per environment.                      |
| RESEND_API_KEY, EMAIL_FROM,<br>EMAIL_REPLY_TO          | Server | Verified sending domain.                            |
| SLACK_WEBHOOK_URL / _ALERTS_URL                        | Server | Ops and alert channels.                             |
| NEXT_PUBLIC_MAP_TILE_KEY,<br>NEXT_PUBLIC_MAP_STYLE_URL | Client | Domain-restricted key.                              |
| CRON_SECRET                                            | Server | <b>SECRET</b> Shared secret for job endpoints.      |
| SENTRY_DSN, SENTRY_AUTH_TOKEN                          | Both   | Errors and source maps.                             |
| NEXT_PUBLIC_APP_URL, VERCEL_ENV                        | Both   | Link generation and environment guards.             |

In the **settings** table, not environment variables: hold duration, SLA hours by tier, deposit percentage, cancellation cut-off and retention schedule, check-in release offset, auto-approval thresholds, quote validity, price-change tolerance, tax rules, minimum turnover hours, maximum stay length before manual handling. Livlux must be able to change these without a deployment.

## Appendix B Booking state transition table

| From                        | To               | Actor           | Side effects                                                                                                                      |
|-----------------------------|------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------|
| DRAFT                       | SUBMITTED        | Partner         | Verify price hash; place hold; create PaymentIntent; notify ops; set <code>submitted_at</code> , <code>decision_due_at</code> .   |
| SUBMITTED                   | UNDER_REVIEW     | Livlux / system | Assign to ops; start SLA clock display.                                                                                           |
| SUBMITTED / UNDER_REVIEW    | EXPIRED          | System          | Release hold; cancel PaymentIntent; notify both sides.                                                                            |
| UNDER_REVIEW                | COUNTER_OFFERED  | Livlux          | New quote; extend hold; notify partner; pause SLA.                                                                                |
| COUNTER_OFFERED             | UNDER_REVIEW     | Partner         | Accept counter; resume SLA.                                                                                                       |
| COUNTER_OFFERED             | CANCELLED        | Partner         | Release hold; void authorisation.                                                                                                 |
| UNDER_REVIEW                | APPROVED         | Livlux          | Live availability re-check; engage exclusion constraint; set <code>ap-proved_at</code> , <code>approved_by</code> .               |
| UNDER_REVIEW                | DECLINED         | Livlux          | Structured reason; release hold; void authorisation; notify with alternatives.                                                    |
| APPROVED                    | CONFIRMED        | System          | Hostaway reservation created; payment captured; invoice issued; hold released; notifications sent; <code>confirmed_at</code> set. |
| APPROVED                    | AWAITING_PAYMENT | System          | Authorisation lapsed or invoice mode; request re-authorisation or await invoice payment.                                          |
| AWAITING_PAYMENT            | CONFIRMED        | System          | Payment received; proceed as above.                                                                                               |
| APPROVED / AWAITING_PAYMENT | FAILED           | System          | Write or capture failed unrecoverably; void authorisation; page ops; hold retained pending human decision.                        |
| CONFIRMED                   | CHECKED_IN       | System          | Arrival date reached; access pack already released.                                                                               |
| CHECKED_IN                  | COMPLETED        | System          | Departure passed; feedback prompt; schedule credential purge.                                                                     |
| CONFIRMED / CHECKED_IN      | CANCELLED        | Either          | Policy refund calculated; Hostaway reservation cancelled; credit note if invoiced; notifications; credentials revoked.            |

Any transition not listed is invalid and must raise `INVALID_STATE_TRANSITION`, be logged as an integrity event, and never be silently absorbed.

## Appendix C Webhook event catalogue

| Source   | Event                                                          | Handler action                                                                                                                                               |
|----------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hostaway | reservation created                                            | Re-fetch reservation → if it is ours, reconcile IDs; if external, refresh the affected calendar span and check for collision with any live platform booking. |
| Hostaway | reservation updated                                            | Re-fetch → detect status, date, price or door-code change → update local mirror → if a confirmed platform booking is affected, alert ops <b>CRIT</b> .       |
| Hostaway | new message received                                           | v1: log and acknowledge. Reserved for a future unified-inbox phase.                                                                                          |
| Hostaway | <i>unknown future event</i>                                    | Log, acknowledge 200, surface on the integrations dashboard.                                                                                                 |
| Stripe   | payment_intent.amount_capturable_updated                       | Mark authorised; release to ops queue.                                                                                                                       |
| Stripe   | payment_intent.succeeded                                       | Mark captured; complete confirmation if the Hostaway write has succeeded.                                                                                    |
| Stripe   | payment_intent.payment_failed                                  | Record failure; notify with a plain-language reason; retain hold for grace period.                                                                           |
| Stripe   | payment_intent.canceled                                        | Release hold; log reason.                                                                                                                                    |
| Stripe   | charge.refunded                                                | Update refund and booking financials.                                                                                                                        |
| Stripe   | charge.dispute.created / .closed                               | Alert finance; freeze automated refunds; assemble evidence pack.                                                                                             |
| Stripe   | invoice.finalized / .sent / .paid / .overdue / .payment_failed | Update invoice lifecycle; drive dunning and credit standing.                                                                                                 |
| Stripe   | payment_method.attached / .detached                            | Sync saved payment methods for the org.                                                                                                                      |

## Appendix D Notification templates

| Template         | Subject line                                               | Must contain                                                                                                                           |
|------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| invite           | You've been invited to the Livlux partner portal           | Inviting org and person, accept link, 7-day expiry, support contact.                                                                   |
| request_received | Request received — {reference}                             | Property, dates, guests, total, claim/PO reference, submission timestamp, stated decision SLA, link.                                   |
| request_approved | Confirmed — {property}, {checkIn} to {checkOut}            | Full booking summary, approver, approval timestamp, invoice link, what happens next, when check-in details arrive.                     |
| request_declined | Unable to confirm — {reference}                            | Reason in plain language, two or three concrete alternatives, direct contact for a human.                                              |
| counter_offer    | Alternative option for {reference}                         | What changed and why, new price, accept/decline actions, expiry.                                                                       |
| payment_failed   | Payment issue — {reference}                                | Plain-language reason, retry link, how long the hold lasts.                                                                            |
| invoice_issued   | Tax invoice {number} — {reference}                         | Amount, due date, PO and claim references, payment link, PDF.                                                                          |
| invoice_overdue  | Invoice {number} is now overdue                            | Amount, days overdue, payment link, consequences stated without threat.                                                                |
| checkin_details  | Check-in details — {property}                              | Exact address, check-in window, access method and code, Wi-Fi, parking, emergency contact, checkout instructions. Marked confidential. |
| hold_expiring    | Your hold on {property} expires in 2 hours                 | What happens on expiry, link to act.                                                                                                   |
| stay_ending      | {guest}'s stay ends in 7 days                              | Extension option with live availability, checkout arrangements.                                                                        |
| ops_new_request  | [Queue] {partner} — {property}, {nights} nights, \${total} | Everything needed to decide, deep link to the queue row, SLA deadline.                                                                 |
| ops_sla_breach   | [SLA] {reference} overdue by {duration}                    | Escalation context and assignee.                                                                                                       |
| ops_critical     | [CRITICAL] {condition}                                     | What happened, affected bookings, link to the relevant runbook.                                                                        |

All partner-facing templates: Livlux branding, plain English, one primary action, full booking reference in both subject and body, plain-text alternative, and a human contact. Never send an automated message about a cancellation or a failure without a way to reach a person.

## Appendix E Glossary

| Term                 | Meaning                                                                                                                                |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Allotment            | Number of units still available on a given day for a multi-unit listing. Governs availability in preference to the day's status field. |
| Authorisation        | A hold on a card's funds without capture. Expires, typically within about seven days.                                                  |
| Capture              | Taking the previously authorised funds. The point at which money actually moves.                                                       |
| Circuit breaker      | A pattern that stops calling a failing dependency for a cooling-off period to avoid compounding the failure.                           |
| Counter-offer        | A Livlux-proposed alternative to a request — different dates, property or price — that the partner may accept.                         |
| Drift                | Divergence between the platform's cached availability and Hostaway's authoritative state.                                              |
| Exclusion constraint | A Postgres constraint preventing rows whose ranges overlap — here, the structural guarantee against double-booking.                    |
| Hard hold            | Blocking dates in Hostaway, removing them from all OTA channels.                                                                       |
| Idempotency key      | A client-supplied identifier making a repeated request safe to replay exactly once.                                                    |
| LOS                  | Length of stay. Drives weekly and monthly discount tiers.                                                                              |
| listingMapId         | Hostaway's channel-mapped listing identifier, required when creating reservations. Distinct from the listing ID.                       |
| Outbox pattern       | Recording the intent to call an external system before calling it, so an ambiguous outcome can be reconciled rather than guessed.      |
| PMS                  | Property management system — here, Hostaway.                                                                                           |
| RLS                  | Row-level security: Postgres policies restricting which rows a role may read or write. The tenancy boundary.                           |
| Soft hold            | Reserving dates within the platform only, leaving OTA channels open.                                                                   |
| SLA                  | The committed window within which Livlux will decide on a submitted request.                                                           |
| Unified webhook      | Hostaway's current webhook mechanism covering reservation created, reservation updated and new message received.                       |

### End of specification

Livlux Partner Direct Booking Platform · Architecture & Build Specification v1.0 · 16 September 2026.

Before build commences, confirm the twelve decisions in §23.2 — item 10 (Hostaway test account) is the critical-path blocker — and re-verify every Hostaway endpoint and field name in §6 against the live API documentation and a real account response.